Automated Generation of Intrusion Detection Rules Using Generative AI in a Rule-Based IDS Deployment

A report submitted in fulfillment of the requirements for the award of the degree of Bachelor of Information Technology

Group 13

Student	ID
Bigyan Siwakoti	S386010
Irab Baral	S387115
Binay Siwakoti	S387596
Shahswat Acharya	S391214

Supervisor: Prof. Bharanidharan Shanmugam

Due date: October 2026

Declaration

I hereby declare that the work herein, now submitted as a final report for the degree of Bachelor of Information Technology at Charles Darwin University, is the result of my own investigations, and all references to ideas and work of other researchers have been specifically acknowledged. I hereby certify that the work embodied in this report has not already been accepted in substance for any degree and is not being currently submitted in candidature for any other degree.

Signature: __binay_bigyan_irab_shahswat_____

Date: [submission date] October 2026

Contents

Contents....3
List of Abbreviations....5
Abstract....6
I. Introduction....7
Background....7
Project Scope....7
Aim of Research....8
Structure of Report....8
II. Literature Review....8
Introduction....8
Intrusion Detection Systems: Taxonomy and Foundations....9
The Rule-Authoring Bottleneck....10
Generative AI for Security Rule Generation....11
Summary....12
III. Approach....12
System Architecture....12
Agentic Loop Architecture....14
Provider Adapter Pattern....14
Tool Design....15
Web Interface....15
Changes from Original Project Plan....16
Summary....16
IV. Execution and Preliminary Results....16
Evaluation Dataset....17
Credential Brute Force....18
Denial of Service and Heartbleed Exploitation....19
Web Application Attacks and Network Infiltration....20
Distributed Denial of Service, Port Scanning, and Botnet....21
Summary....22
V. Analysis and Discussion....23
Introduction....23
Model Comparison....23
AI-Authored Rules vs Stock Community Rules....24
Raw vs Suppressed Data....25
Cross-Day Generalization....26

Model Errors and Rule Provenance....26
Summary....27
VI. Summary....27
VII. Future Work....28
References....30
Appendix A: Turnitin Similarity Report....32

List of Figures

Fig. 2-1 : IDS taxonomy diagram showing signature-based, anomaly-based, and specification-based approaches with their characteristics

Fig. 3-1 : System architecture diagram showing Suricata → eve.json → AI Triage Engine → Detection Rules / Suppress Proposals, with the Web UI for real-time streaming

Fig. 3-2 : Safety asymmetry — detection rules deploy automatically after validation; suppressions require human approval

Fig. 3-3 : Agentic tool-calling loop — every interaction between the hosted model and the sensor passes through the engine and a validated tool

Fig. 3-4 : The agent's toolkit grouped by purpose, with the engine-enforced safeguards

Fig. 3-5 : Web UI screenshot showing model selection dropdown, run selector, and live triage streaming panel

Fig. 3-6 : Experiment profiles — one named, frozen bundle of configuration, prompt and tools per condition (the UNSW v1 profile, which predates profiles, is omitted)

Fig. 3-7 : The hunt condition — the detection-engineering lifecycle as the agent's procedure, with an engine-enforced test gate

Fig. 3-8 : test_rule — a topology rule fails when widened to any → any; a behavioural rule passes (capture 3)

Fig. 4-1: Web UI screenshot showing the triage engine during a live investigation, with tool calls and model reasoning visible in the streaming panel (illustrative run captured for this figure, not part of the evaluation set)

Fig. 4-2 : Alert volume per capture — CIC-IDS2017 tuned and raw passes, UNSW-NB15 raw (log scale)

Fig. 4-3 : Ground-truth categories identified by each UNSW-NB15 triage run

Fig. 4-4 : UNSW-NB15 TCP/UDP detection as rulesets and disabled rule families are added

Fig. 4-5 : Selected families versus enabling every disabled rule — detection and alerts on non-attacker hosts

Fig. 4-6 : Baseline v2 detection by UNSW-NB15 attack category

Fig. 5-1 : CIC-IDS2017 — labelled attack flows matched by stock ET Open and by stock plus AI-authored rules

Fig. 5-2 : UNSW-NB15 — TCP/UDP ground-truth attack events matched by stock ET Open and by stock plus AI-authored rules

Fig. 5-3 : Share of label-matched alerts that fall on attack flows — stock ET Open versus AI-authored rules

Fig. 5-4 : Attack categories covered by stock rules, by stock plus AI-authored rules, and by the model's triage findings

Fig. 5-5 : Status of every AI-authored rule definition

Fig. 5-6 : AI-round gains under the old metric, by attribution, and with topology rules removed

Fig. 5-7 : Classes of AI-authored rules by condition

Fig. 5-8 : Protocol records by source role on capture 3 — the gate can only test protocols used by internal hosts

Fig. 5-9 : Alerts of each hunt rule on each development capture, grouped by the capture it was written on

Fig. 5-10 : Genuine gain on all development captures, on development captures without rules, and on the hold-out

Fig. 5-11 : Genuine development events found by the hunt and hunt2 conditions, and each condition's top rules

Fig. 5-12 : Baseline v2 detection against all, content-detectable and ever-detected attack events

Fig. 5-13 : Attack-like activity the hunt reports left without a rule, grouped by cause

Fig. 5-14 : Frozen baseline v2 on the development captures and on the hold-out captures

Fig. 5-15 : Experiment 1 — breadth of detection and depth of outcome evidence with full sensor context and with alerts only

List of Tables

Table 0-1: Glossary of project terms

Table 3-1: The agent's tools — the only actions the model can take

Table 3-2: Tools and software used

Table 4-1: CIC-IDS2017 dataset structure ,captures, attack categories, and ground-truth flow counts.

Table 4-2: Execution summary — alert volumes, attack categories identified and AI-authored rule definitions per capture

Table 4-3: UNSW-NB15 triage runs (DeepSeek V4 Pro, raw captures)

Table 4-4: Challenges of UNSW-NB15 and how they were handled

Table 4-5: Strengthening the UNSW-NB15 baseline (six development captures)

Table 4-6: AI rounds on baseline v2 — events and percentage points of the 14,935 TCP/UDP attack events; rules counted over both rounds

Table 4-7: Hunt and hunt2 runs (DeepSeek V4 Pro, web interface)

Table 4-8: Rules written in the hunt condition (captures 3, 1 and 10); every rule passed test_rule before it was saved

Table 5-1: Model comparison across triage runs. Turn counts are from session transcripts.

Table 5-2 -Detection coverage: AI rules vs stock ET Open

Table 5-3: Audit of AI-authored rule definitions

Table 5-4: Frozen AI rules on the hold-out (captures 1, 3, 6: 4,634 TCP/UDP events; benign-only captures 25, 50)

Table 5-5: Detection against all, content-detectable and ever-detected attack events

Table 5-6: Experiment 1 — full sensor context versus alerts only (DeepSeek V4 Pro, UNSW-NB15)

Table D-1: Inventory of AI-authored rule definitions

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
C2	Command and Control
CIC	Canadian Institute for Cybersecurity
CVE	Common Vulnerabilities and Exposures
DDoS	Distributed Denial of Service
DoS	Denial of Service
ET	Emerging Threats
FTP	File Transfer Protocol
HIDS	Host-based Intrusion Detection System
IDS	Intrusion Detection System
IDPS	Intrusion Detection and Prevention System
LLM	Large Language Model
ML	Machine Learning
NDJSON	Newline-Delimited JSON
NIDS	Network-based Intrusion Detection System
NIST	National Institute of Standards and Technology
pcap	Packet Capture
SID	Signature Identifier
SMB	Server Message Block
SOC	Security Operations Centre
SSH	Secure Shell

SSE	Server-Sent Events
TLS	Transport Layer Security
XSS	Cross-Site Scripting
GT	Ground Truth
UNSW	University of New South Wales

Glossary of Project Terms

Terms coined or given a specific meaning in this project.

Term	Meaning in this report
Ground truth (GT)	The dataset's own labels of which traffic is an attack (CIC-IDS2017 labelled flows; UNSW-NB15 NUSW-NB15_GT.csv events). Used only for scoring; the model never sees it.
Development (dev) set	The six UNSW-NB15 captures from 17 February 2015 (1, 3, 4, 5, 10, 14) on which rule families were chosen and AI rules were written and scored.
Hold-out	UNSW-NB15 captures from 22 January 2015 (1, 3, 6 for detection; 25, 50 benign-only) kept untouched until every rule file was frozen, then scored once.
Baseline v2	The strengthened human-written rule set: ET Open plus Snort Community, Stamus lateral, PT Open and the ET/Snort rule families that ship disabled, selected by the services seen in the traffic. Frozen; contains no AI rules.
All-rules control	Baseline v2 with every disabled rule enabled; used to show the cost in noise of not selecting families.
Experiment profile	A named, frozen bundle that fixes a run's dataset, Suricata configuration, writable rule file, tools, prompt and blocked files (src/profiles.py).
Plain condition	Profile unsw-v2-plain: alert triage on baseline v2 with the search_rules tool.
Research condition	Profile unsw-v2-research: plain plus lookup_vulnerability, a local copy of the NVD database.
Hunt condition	Profile unsw-v2-hunt: the detection-engineering workflow (coverage map, rare-value hunting, hypothesis, rule with ADS comment, test_rule, save), without the attacker range in the prompt.
Hunt2 condition	Profile unsw-v2-hunt2: the hunt condition plus the suricata_keywords tool; otherwise identical.
Round (R1, R2)	One pass of rule writing over captures 3, 1 and 10 in turn, followed by scoring on all six development captures; a second round starts from the first round's rules.
stack_values	Tool that counts the values of any eve.json field and lists the rarest first (stack counting / long-tail analysis).
test_rule (the gate)	Tool that checks a rule's syntax, replays a capture with only that rule, then replays it with its addresses widened to any → any. FAIL if it raises no alerts or if the widened rule fires between internal hosts. The engine refuses to save a rule that has not passed in exactly that form.
suricata_keywords	Tool (hunt2 only) that lists the rule keywords the installed Suricata 8.0.6 supports, with descriptions and documentation links.
Known-good traffic	Traffic between internal hosts (59.166.0.0/24 clients to 149.171.126.0/24 servers), treated as normal on the basis of the dataset's published testbed description; the gate's reference.
Topology rule	An AI rule that matches ordinary protocol activity from outside HOME_NET with no attack constraint (e.g. SMTP MAIL FROM, POP3 login). It scores well on UNSW-NB15 only because every external host there is an attacker.
Signature rule	A rule keyed on attack content (exploit strings, traversal, format strings, injection).
Behavioural rule	A rule keyed on a rate or on protocol misuse (e.g. over-long fields, fuzzed file names, parser anomalies) rather than a fixed string.
Dataset-indicator rule	A rule keyed on a string that identifies this laboratory rather than an attack technique (e.g. a test domain name).
Instance rule /	An instance rule matches one exploit attempt's exact payload (e.g. its

technique rule	filler bytes) and rarely fires again; a technique rule matches the property every attempt shares (e.g. any ../ traversal).
Replay variance	Two Suricata replays of the same capture with the same rules do not produce identical alerts, because thresholds and flow handling depend on timing.
Attributable gain	Attack events matched by an AI-rule alert and by no other alert in the same replay.
Genuine gain	Attributable gain counted from signature and behavioural rules only — topology and dataset-indicator rules excluded.
Content-detectable event	An attack event in which the attacker sent application data in a matching flow, so a content rule could in principle see it.
Ever-detected event	An attack event caught by at least one configuration run in this project (baseline, all-rules control or a genuine AI rule); a practical, circular ceiling.
Thinking mode / reasoning carry	DeepSeek V4 Pro may return its reasoning in a separate reasoning_content field. Until a fix made during Experiment 3 the adapter discarded it, so the model's reasoning was not carried to the next turn; it now is.
ADS comment	A comment above each hunt rule stating its goal, blind spots and expected false positives, after Palantir's Alerting and Detection Strategy framework.

Table 0-1: Glossary of project terms

Abstract

The network intrusion detection systems need human analysts to review alerts, create detection rules, and configure the sensor settings. This burdensome process introduces a structural delay between identifying and detecting an attack. The purpose of this

project is to explore the ability of a generative language model to eliminate this analyst task in a rule-based IDS deployment while keeping the signature-based detection deterministic and easily inspected, but significantly reducing the amount of human effort needed.

We introduce an agentic triage engine which integrates LLM APIs with Suricata IDS, enabling the model to use investigative capabilities to query alert data, correlate network flows, author detection rules, and validate alerts with engine replay. The system was tested on the dataset from CIC-IDS2017 with three model configurations on all four days of the attacks, namely Claude Opus 5, DeepSeek V3, and DeepSeek V4 Pro. It was then evaluated, unchanged, on a second dataset it had never been tuned for (UNSW-NB15), and every AI-authored rule was scored against both datasets' ground-truth labels. On CIC-IDS2017 the model identified 13 of 14 labelled attack categories, and its rules raised the number of categories caught by at least one true-positive alert from 8 (stock ET Open alone) to 12. On the two noisiest captures, 99.4% of label-matched AI-rule alerts fell on attack flows, against 6.1% for the stock rules. On UNSW-NB15 the model identified 8 of 9 attack categories across six captures, and its rules raised event-level detection of TCP/UDP attacks from 9.2% to 15.5% without adding a single alert on a benign host. The evaluation also documents consequential model errors: deleting effective rules on a hallucinated claim, writing rules that do not load, and reading routine traffic as data exfiltration. A context experiment showed that the model identified attacks as broadly from alert events alone, but only full sensor context let it establish whether those attacks succeeded. A second phase strengthened the human baseline the way a detection engineer would — re-enabling rule families that ship disabled, chosen from the services seen in the traffic — which raised UNSW-NB15 TCP/UDP detection from 9.2% to 25.7%. Against that baseline, three AI conditions were compared: two triage-driven conditions added genuine detection of only +0.08 and +0.33 percentage points, most of their apparent gain coming from rules that match the dataset's network layout rather than attack behaviour, while a condition that replicates detection-engineering practice with an anti-topology test gate added +1.21 points, and +1.53 on untouched hold-out captures with no alert on benign-only

traffic. That gain coincides with a fix to how the model's reasoning is carried across turns, made during the round, and the two effects cannot yet be separated. Adding a reference of supported rule keywords removed rule-loading failures but did not raise detection (+1.06 points on the hold-out): different runs of the same procedure found largely different attacks. Most missed attacks carried data, so the remaining gap is unwritten signatures rather than undetectable traffic. The frozen baseline alone detected 29.5% of hold-out attack events.

Keywords: intrusion detection system, generative AI, Suricata, rule generation, CIC-IDS2017, agentic triage, UNSW-NB15, ground-truth evaluation

I. Introduction

Background

IDP systems (IDS) are a fundamental part of network security infrastructure that monitor network traffic and detect signs of malicious activity. There are a variety of approaches to intrusion detection, and the signature-based approach is still the most widely used open-source deployment model today. When correctly configured, these systems can compare network traffic against a database of human-written rules, so they will behave deterministically, provide inspectable alerts, and have a low false-positive rate (Roesch, 1999; Khraisat et al., 2019).

This has one structural problem, however: if no rule is defined, the attack will not be detected, even if the detection engine is highly advanced. The detection capability of a signature-based IDS is limited solely by the rules it can have. This dependency was discovered as early as computer

security threat monitoring studies by Anderson (1980), and is the main weakness of the misuse detection systems today (Khraisat et al., 2019).

Manually writing a detection rule for a new attack, whether it's discovered during alert investigation, threat intelligence, or incident response, is a process that always happens. To create a rule, an analyst needs to analyze the traffic, find distinguishing characteristics, write a rule in the syntax of the engine in question, test the rule against both attack and benign traffic, and deploy the rule to production sensors. Scarfone and Mell (2007) state that this cycle can take hours to days and is a specialty that is rare in the cybersecurity profession. This choke point is a major operational risk, especially for organizations that are short-staffed in their security operations center (SOC).

This project explores the possibility of replacing the human analyst in the alert-handling loop with a generative language model, but not the detection engine itself. Rule-based detection is still fully present, and it can be inspected throughout; rules that the deterministic engine evaluates.

Project Scope

For this project, Suricata 8.0.6 is implemented in offline packet capture (pcap) replay mode against the CIC-IDS2017 dataset (Sharafaldin, Habibi Lashkari & Ghorbani 2018) and, for cross-dataset evaluation, the UNSW-NB15 dataset (Moustafa & Slay 2015). The AI triage engine can connect to hosted model APIs and includes tools for questioning alert data, reading configurations, writing detection rules, and running Suricata for validation. Characterization of traffic is not done directly in the model; rather, rules are authored for the deterministic engine to evaluate. By design, machine learning is not included in this project, since the project is based on a commitment to inspectable, rule-based detection.

Aim of Research

The main research question is: Can an LLM be used to take over the role of the human analyst in an alert-handling workflow of a rule-based IDS, and what context does it require to do a good job of it?

The project aims to achieve two goals to answer this question:

- Create an agentic triage engine that equips a generative language model with the tools and guidance to complete the analyst workflow: from initial alert evaluation to rule writing and rule validation.

- Measure the detection capability for various attack categories by comparing AI rules with the stock community ruleset to determine coverage gaps closed, and comparing the results from different models. Both are measured against the datasets' ground-truth labels, including on a second dataset the system was never tuned for.
- Establish what context the model needs, by comparing triage given only alert events with triage given full sensor context (Experiment 1).

Structure of Report

The rest of this report is structured in the following way:

- Section II reviews the relevant literature and analyses the current intrusion detection system (IDS) market, the rule-authoring problem, and the recent application of generative AI in security operations.
- The approach used, along with system architecture and design decisions, is detailed in Section III.
- Section IV explains how the triage engine was implemented over the four attack days of the CIC-IDS2017 dataset the results, and the cross-dataset evaluation on UNSW-NB15. It also reports the strengthened UNSW-NB15 baseline and the AI rule-writing rounds run against it (Experiment 3).
- The findings are analysed and discussed in Section V, with a comparison of the models, as well as the quality of the rules and the meaning of the raw versus suppressed alert data. It also presents the ground-truth evaluation of every AI-authored rule, the rule audit, the context experiment, and the threats to validity identified during evaluation.
- A summary of the work and its conclusions is included in Section VI.
- Future work is described in Section VII.

II. Literature Review

Introduction

This review will cover three areas related to IDSs: the taxonomy and theory of IDSs, the knowledge-acquisition issue in the case of rule-based systems, and the nascent field of generative AI for security rule authoring. The review is organized by theme, starting with some basic concepts, then the limitations that drove this project, and then recent work that is directly

relevant to this approach. The goal is to set the stage for this project: To use generative AI not just for rule generation, but for the entire analyst workflow in a network IDS deployment.

Intrusion Detection Systems: Taxonomy and Foundations

The formalization of the concept of automated intrusion detection happened first in 1987 with the publication of the model of real-time detection put forward by Denning that monitored audit records for deviations from user profiles. This groundbreaking study set the standard for the two major methods used today: misuse detection, where observed activity is compared to known attack patterns, and anomaly detection, where deviations from a baseline of normal activity are identified.

According to Khraisat et al. (2019), there are three types of intrusion detection systems: signature-based (misuse detection), anomaly-based, and specification-based. Signature-based systems like Snort (Roesch, 1999) and the multi-threaded Snort variant Suricata compare network traffic to a database of known attack patterns. The key strength of their approach is determinism, the ability of each alert to refer to a human-readable rule, which is their preferred option for an environment where auditability and forensic clarity are key (Scarfone & Mell, 2007). Paxson (1999) confirmed this by implementing a real-time semantic analyzer for network traffic at operational scale using the Bro network monitor (now Zeek), which could be used for both signature matching and policy-based detection.

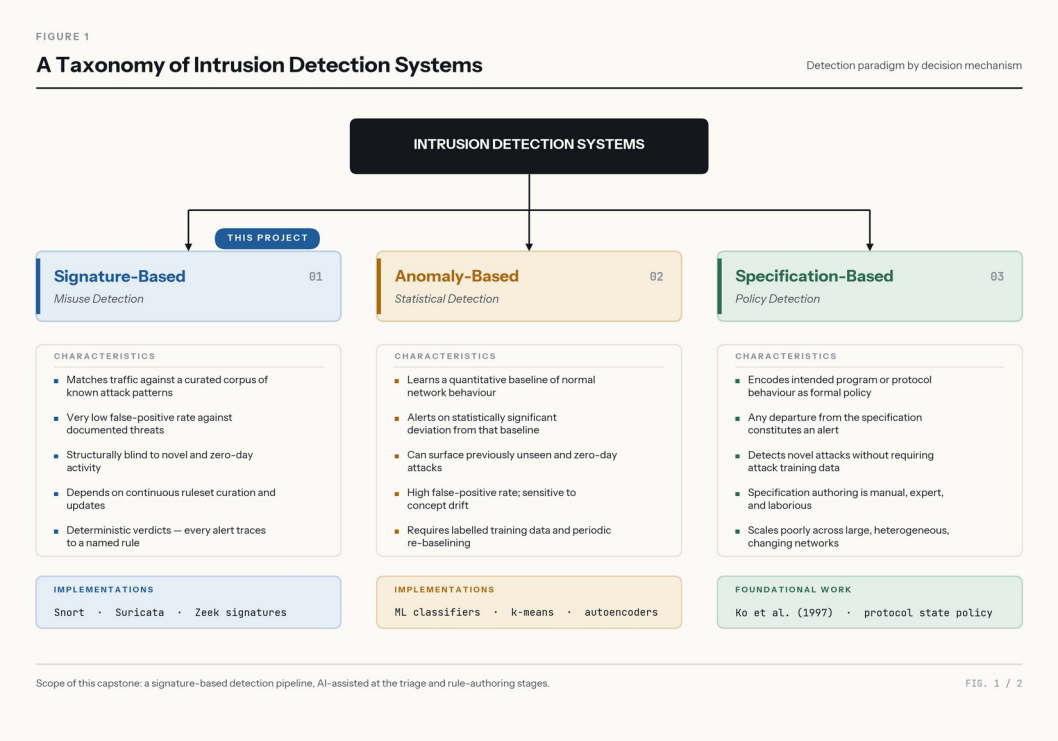


Fig. 2-1 : IDS taxonomy diagram showing signature-based, anomaly-based, and specification-based approaches with their characteristics

Anomaly-based systems define "normal" and highlight any anomalies. They can also identify novel attacks, but have high false-positive rates and make classifications that are hard to interpret or audit (Liao et al., 2013; Garcia-Teodoro et al., 2009). This interpretability gap is important in regulated settings where the response to incidents should include reasoning chains that are demonstrated. Aljanabi, Ismail, and Ali (2021) agree that this is a longstanding problem, as even though classifiers for machine learning have improved, the lack of explanation of the classifiers' decisions is an obstacle to operational deployment. Axelsson (2000) also showed the base rate fallacy problem: even if the anomaly detector is very accurate, it will generate too many false alarms when the base rate of intrusions is low compared to normal traffic for the system to be used in operations.

In their paper, Sommer and Paxson (2010) have done an extensive study on the reasons machine learning has failed to achieve widespread adoption in the field of network intrusion detection: the lack of representative data for training, the high cost of misclassifications in the security domain, and the semantic gap between statistical anomalies and security alerts. The use of automated techniques has been suggested to aid human analysts rather than totally replace them, and this is the design philosophy for this project.

This is called specification-based detection, which was first formalized by Ko, Ruschitzka, and Levitt (1997), and involves explicitly specifying acceptable behaviour in the system and identifying deviations from the specification. Sekar et al. (2002) extended this to network intrusion detection, showing that specification-based approaches can get low false-positive rates from signature detection and have some capability of detecting novel attacks that violate specifications. The maintenance of the specification, however, is labor-intensive as the specification has to be updated every time the system being protected changes, which has limited practical application.

The Rule-Authoring Bottleneck

This signature-based IDS limitation has been discussed extensively in the literature. It was found that the reliance on human-written signatures is the main drawback: zero-day attacks and new variants slip through until a new rule is created (Khraisat et al., 2019). Scarfone and Mell (2007) stated that the analyst workflow of traffic analysis, invariant identification, rule authoring, syntax validation, behavioral testing, and deployment is a multi-hour to multi-day process with a need for special knowledge of attack techniques and engine rule syntax.

This restriction is of a knowledge-acquisition nature. The analyst has an implicit knowledge of how to determine the difference between attack traffic and legitimate traffic and must make this knowledge explicit by converting it into the syntax of the detection engine. It is a labor-intensive challenge, not a technical one, between an attack being identified and detected, which is a huge danger in resource-challenged situations. Buczak and Guven (2016) reported that the lack of domain knowledge is one of the major challenges in the effective deployment of IDS, irrespective of the IDS detection method used in their survey of machine learning for cyber intrusion detection.

Partially, Proofpoint's Emerging Threats (ET) Open ruleset uses a community-based rule development process to offer more than 52,000 rules that are focused on known attack signatures. Community rulesets are, however, reactive; they target known attacks in the wild, and there are gaps in coverage for certain variants of attacks and for certain deployments, as witnessed by our preliminary results, in which the community ruleset did not cover a whole category of attacks in the evaluation set.

Generative AI for Security Rule Generation

With the advent of large language models (LLMs) capable of producing structured text, there's an opportunity to automate tasks in the security

operations space that were once thought to be uniquely human. This project uses the transformer architecture of modern LLMs to process and generate formally structured outputs, such as intrusion detection rules, in this instance.

Recently, efforts have started to investigate generative language models for security rule generation in particular. Recently, researchers have started studying generative language models for security rule generation specifically. In Zhang et al. (2025), detection rules are automatically generated for malicious software packages using LLM and achieve syntactically valid rules, which are able to detect known malware patterns. They did not work on network-level detection rules, but rather on package-level signatures, and concluded that, with specific context and examples, the LLMs can create formally structured rules for security.

To evaluate the performance of LLM in autonomous IDPS rule generation for unseen attacks, Hadi, Yasmin, and Shoker (2026) directly addressed the issue of IDS rule generation by using GenTI. Their benchmark revealed that baseline LLM-generated rules were effective at catching 45% of previously unseen attacks, increasing to 87.4% when they mapped out CTI and iteratively refined their rules. GenTI produces rules from a curated payload corpus; in this project, the model is allowed to have investigative access to the sensor output such that it can derive rules from the traffic patterns it observes during the triage phase in lieu of pre-prepared threat descriptions.

In a survey of deep learning techniques in cybersecurity, Ferrag et al. (2020) classified the techniques into intrusion detection, malware analysis, and network traffic classification. Their work, which was concerned with detection rather than rule generation, however, highlighted the problems with just an automated method, especially the requirement for interpretation of the results, and the difficulty of dealing with new attack patterns, which will be a challenge for this project since the model will remain in an advisory and authoring capacity.

A known risk of generative models is hallucination: fluent, confident output that is not supported by the input or by fact (Ji et al. 2023). In a conventional chatbot this produces a wrong answer; in an agent that can write and delete detection rules it produces a wrong action. Section V documents a case of exactly this kind, which is why the system keeps a human in the loop for any change that reduces detection.

The agentic paradigm is a stark contrast to previous attempts at AI-powered security, in which a language model was provided with a set of tools and then tasked with using them sequentially to accomplish a set of

objectives. The agentic paradigm significantly shifts the way that an AI model operates in security use-cases, as opposed to the previous paradigm, where the language model was given one set of tools and one set of objectives, and then tasked to use the tools one at a time and in sequence to achieve those objectives. In contrast with the description-to-rule paradigm, in the agentic approach, the agent gives the model the same investigative power as a human analyst, and the model investigates on its own before writing the rules. This is based on the general trend of AI system designs from prompt-response to tool-augmented reasoning (Anthropic, 2025). The design follows the ReAct pattern, in which a language model interleaves reasoning with actions against an external environment and uses each observation to plan its next step (Yao et al. 2023), and builds on evidence that language models can learn when and how to call external tools (Schick et al. 2023).

Evaluation Datasets and Label Quality

Evaluating an intrusion detection system requires labelled traffic. Ring et al. (2019) survey the available network datasets and show that they differ widely in realism, labelling method and attack coverage, so results on a single dataset generalise poorly. CIC-IDS2017 (Sharafaldin, Habibi Lashkari & Ghorbani 2018) labels every flow produced by CICFlowMeter; UNSW-NB15 (Moustafa & Slay 2015, 2016) was generated with the IXIA PerfectStorm tool and lists each attack event with its endpoints, time and one of nine attack categories. Using both allows the approach to be tested beyond the dataset it was developed on.

Labels are not error-free. Engelen, Rimmer and Joosen (2021) identified flow-construction and labelling problems in CIC-IDS2017, and Lanvin et al. (2023) documented packet misordering, duplication and attacks that were performed but not correctly labelled. This matters for the present evaluation: where an AI-authored rule and the labels disagree, the disagreement has to be examined before it is counted as an error of the rule.

Detection Engineering Practice and LLM-Generated Rules

In practice, detection rules are produced by a detection-engineering lifecycle rather than written ad hoc: a hypothesis about attacker behaviour is turned into a rule, the rule is tested against traffic known to contain the behaviour and against traffic known to be benign, reviewed, deployed and tuned. Palantir's Alerting and Detection Strategy (ADS) framework formalises this as a template that every detection must complete before release, with sections including the goal, the technical

context, blind spots and assumptions, expected false positives and validation (Palantir 2017). Atkinson (2020) argues that rules should target the abstract capability an attacker needs rather than the surface features of a particular tool, because tools change while the underlying technique does not. Threat hunting supplies the hypotheses: stack counting, or long-tail analysis, counts every value of a field and examines the rare ones, which needs no statistical model and no training period (Dragos 2023).

Several recent systems use large language models to generate intrusion detection rules. FALCON generates Snort and YARA rules from cyber-threat-intelligence reports and filters them through syntactic, semantic and performance validators (Mitra et al. 2025). GRIDAI uses several cooperating agents to decide whether an attack sample is new, in which case a rule is written, or a variant of a known attack, in which case an existing rule is repaired (Li et al. 2025). GenTI benchmarks models on IDPS rule generation (Hadi, Yasmin & Shoker 2026). RuleAutoPilot is the closest to this project: an agentic pipeline that writes Suricata rules from malware traffic, filters known-benign flows first, and accepts a rule only if it loads, fires on the source traffic and stays silent on a benign corpus; this execution-grounded validation raised its F1 score from 0.443 to 0.539 (Rehman, Piplai & Kantarcioglu 2026). A user study with ten domain experts found that experts treat LLM-written network rules as drafting and verification aids rather than autonomous generators, citing low specificity and logic errors in 12% of cases (di Filippo et al. 2026).

These systems start from labelled malicious samples or threat intelligence. This project asks a different question: whether an agent working from unlabelled sensor traffic, with no access to ground truth, can add detection to a strong human-tuned baseline — and it borrows the validation idea from RuleAutoPilot and the procedure from detection-engineering practice to test it (Section III).

Summary

The literature confirms that rule-based detection works but is limited by the requirement for human authoring, while machine-learning alternatives lack the interpretability required for operational environments. Generative AI for rule authoring is a new, emerging area of research with proven potential, but has limited research specifically for network IDS triage. This project fills a gap to use generative AI not just for rule generation, but for the entire analyst workflow (from investigation to correlation to decision making to rule authoring) in a network IDS deployment. This work differs from previous ones that reasoned about generating rules by adopting the rule-based approach or the model-based approach, both of which considered the task as a single-shot translation.

III. Approach

Here we explain the system architecture, the design decisions that guided the implementation, and the modifications introduced from the initial project plan as our understanding of the system grew during iterative development.

System Architecture

The system follows pipeline-style design with distinct layers for the detection engine and the AI triage layer.

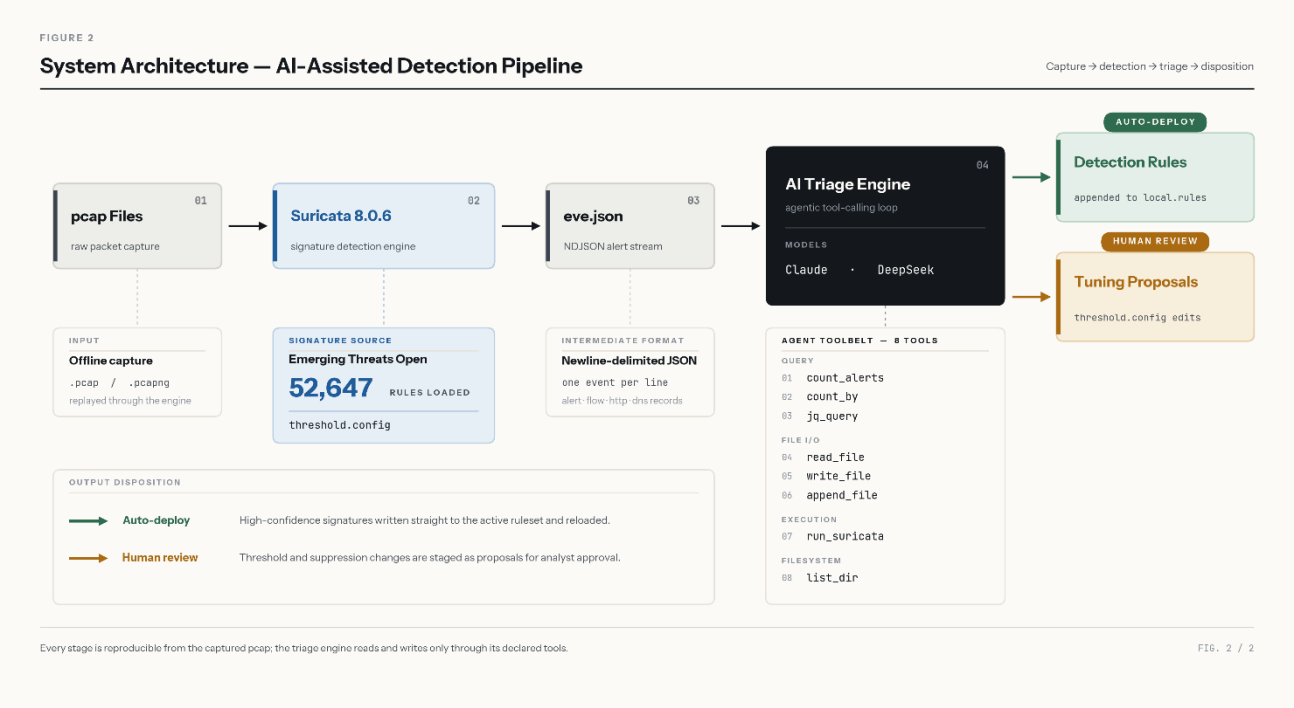


Fig. 3-1 : System architecture diagram showing Suricata → eve.json → AI Triage Engine → Detection Rules / Suppress Proposals, with the Web UI for real-time streaming

The system works on 3 major stages, which are explained below:

Stage 1: Detection: Suricata 8.0.6 (OISF 2026) reads pcap files and applies the ET Open community ruleset (Proofpoint, 2026) containing 52,647 rules, and outputs all the events (alerts, DNS queries, TLS handshakes, HTTP transactions, and flow records) to a newline-delimited JSON (NDJSON) file called eve.json. All benign traffic is processed (CIC-IDS2017 Monday 529,918 flows, all labeled BENIGN), and all false positives are suppressed to establish a baseline. This resulted in 80 documented suppress rules in threshold.config, as in real SOC practice. Once tuned, no alerts are generated on Monday.

Stage 2 : Triage: The alert counting, NDJSON aggregator with a custom count_by function, arbitrary jq queries against eve.json, file reading, file

writing, file appending, rule validation with Suricata, and directory listing are all supported by the AI triage engine, which processes alerts on the day of the attack. The model is based on a triage procedure with six steps in the system prompt - establish the big picture, categorize alerts, investigate each category, deep-dive into specific flows, take action, and report findings.

Stage 3: Output: The model yields two types of outputs with a safety asymmetry. If a threat is confirmed, the model generates Suricata detection rules to a specified directory, and they are deployed automatically. The model suggests suppressions in case of false positives, which have to be accepted by humans before implementation. This asymmetry makes the model wrong-way-safe when it comes to silently degrading sensor coverage: an over-alerting IDS is wrong-way-safe, a blind IDS is wrong-way-unsafe.

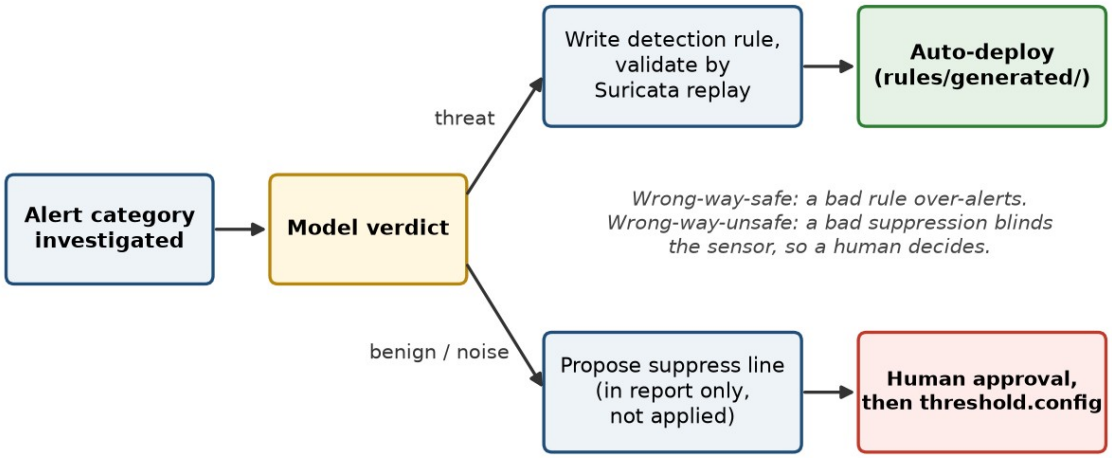


Fig. 3-2 : Safety asymmetry — detection rules deploy automatically after validation; suppressions require human approval

Agentic Loop Architecture

At their heart, the hosts are text-generation models, meaning that they take an input prompt and output a sequence of text. They do not have any inbuilt capabilities to check the databases, to read files, or to run programs. Such a model is transformed into an autonomous investigative agent through a tool-calling loop that mediates its relationship with the operating environment.

The operation of the loop is as follows. Each turn, the engine passes the conversation history to the model along with a schema for each available tool, including its name, parameters, and use. The model can send a

structured tool-call request, such as count_by(eve_path="logs/tuesday/eve.json", field=".alert.signature"), rather than just a plain-text response. The engine checks the request, runs the function in Python, and adds the new result to the conversation as a tool-result message. Then, the model examines that result and determines what to do next, whether it is another tool call, reasoning about the result, or creating a final report. The cycle continues until the model responds with a text-only message (when it is finished) or a limit on the number of turns (that can be programmed).

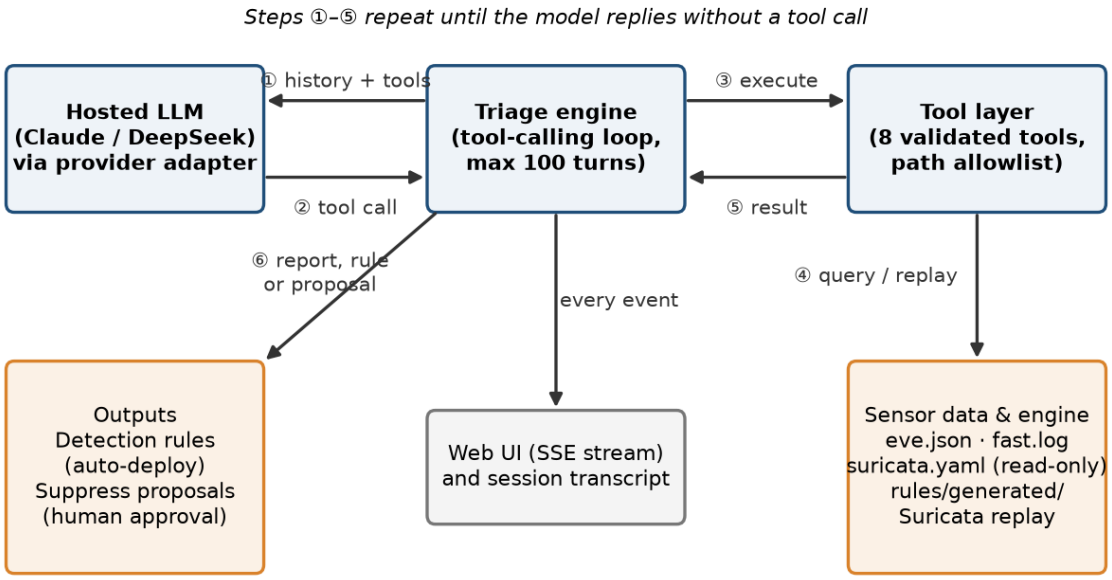


Fig. 3-3 : Agentic tool-calling loop — every interaction between the hosted model and the sensor passes through the engine and a validated tool

This architecture is important for two reasons. It preserves the model's reasoning capabilities and provides controlled access to real data — the model determines what it should investigate and in what order, but all access to the filesystem and the detection engine is through a validated tool interface. Second, it allows for the system to be model agnostic, allowing any model that supports tool calling to trigger the same investigation loop, comparison across providers, without rewriting the triage logic.

The system prompt provides a six-step triage process (big picture, categorize alerts, investigate each category, deep dive into specific flows, take action, report findings) to guide the use of tools by the model. This procedural guidance outlines the investigation process, with the model operating in a manner similar to that of a human investigator.

Provider Adapter Pattern

A provider adapter pattern is introduced to enable the same triage loop to support Claude (Anthropic API) and DeepSeek (OpenAI-compatible API), allowing fair comparison between the models. Elevated alerts are identical across both providers, as are the tool definitions and triage processes. The adapter can convert between the two API formats without the agentic engine knowing, which then operates without knowing which model is running the investigation.

Tool Design

It turns out that the design of the tools is as important as the choice of the model. On initial triage runs, DeepSeek V3 has failed to work due to using aggregation operations (group_by, unique, min_by, max_by) that are incompatible with the NDJSON streaming format. The model ran out of turns before completing triage. The answer was a custom-made count_by tool that aggregates in Python using collections.Counter, so it is not affected by the NDJSON limitation. DeepSeek successfully performed triage after this addition. This shows that a lower-cost model with more suitable tools can perform as well as a more capable model using more generic tools.

A hosted language model can only produce text. What makes it an agent is the set of tools the engine offers it: each tool is described to the model as a JSON schema (name, purpose, parameters), the model replies with a structured request to call one, and the engine — not the model — checks the request against the experiment profile, runs it and returns the result for the next turn (Figure 3-3). The model can therefore investigate, research, validate and write rules, but only through these 13 functions, each with its own limits (Table 3-1, Figure 3-4). Eight were built for the first CIC-IDS2017 triage; five were added for the UNSW-NB15 experiments.

Tool	What it does	Safeguards	Profiles
count_alerts	Alerts in an eve.json grouped by signature, most frequent first — the usual first step.	Allowlisted paths only.	All
count_by	Counts the values of any jq expression over eve.json (event types, IP pairs, protocol fields), in Python, so it works on NDJSON.	Allowlisted paths; 120 s timeout.	All
query_eve_json	Runs a jq filter over eve.json and returns matching events.	150-line cap with a hint to aggregate; 60 s timeout.	All
read_file	Reads a configuration, rule or log file.	Only logs/, config/, rules/, data/pcaps/; labels, notes, transcripts and results refused; 150-line cap.	All
list_directory	Lists files with sizes.	Blocked paths hidden.	All
write_file /	Writes a detection rule file, or proposes	Only rules/generated/ and	All

append_to_file	a suppression.	threshold.config; suppressions need human approval; hunt profiles: append only, own rule file only, and only rules that passed test_rule unchanged.	
run_suricata	Replays a capture with the profile's Suricata configuration into logs/<run>/, so the model can validate its rules.	Profile's configuration only; 10-minute timeout.	All
search_rules	Searches the rules actually loaded (message, references, content) — 'is this already covered?'	Read-only.	Plain, research, hunt, hunt2
lookup_vulnerability	Product, flaw type and description from a local copy of the NVD CVE database (2002–2016).	Offline; refuses dataset or traffic-generator terms.	Research, hunt, hunt2
stack_values	Counts a field's values and lists the rarest first — stack counting for hunting.	Allowlisted paths; capped list.	Hunt, hunt2
test_rule	Syntax check, replay with only the rule, replay with addresses widened to any → any; reports alerts by host role and a PASS/FAIL verdict per rule.	Reports counts only, never labels; scratch output deleted after each test.	Hunt, hunt2
suricata_keywords	Lists the rule keywords this Suricata build supports; exact names return the documentation link.	Read-only.	Hunt2

Table 3-1: The agent's tools — the only actions the model can take

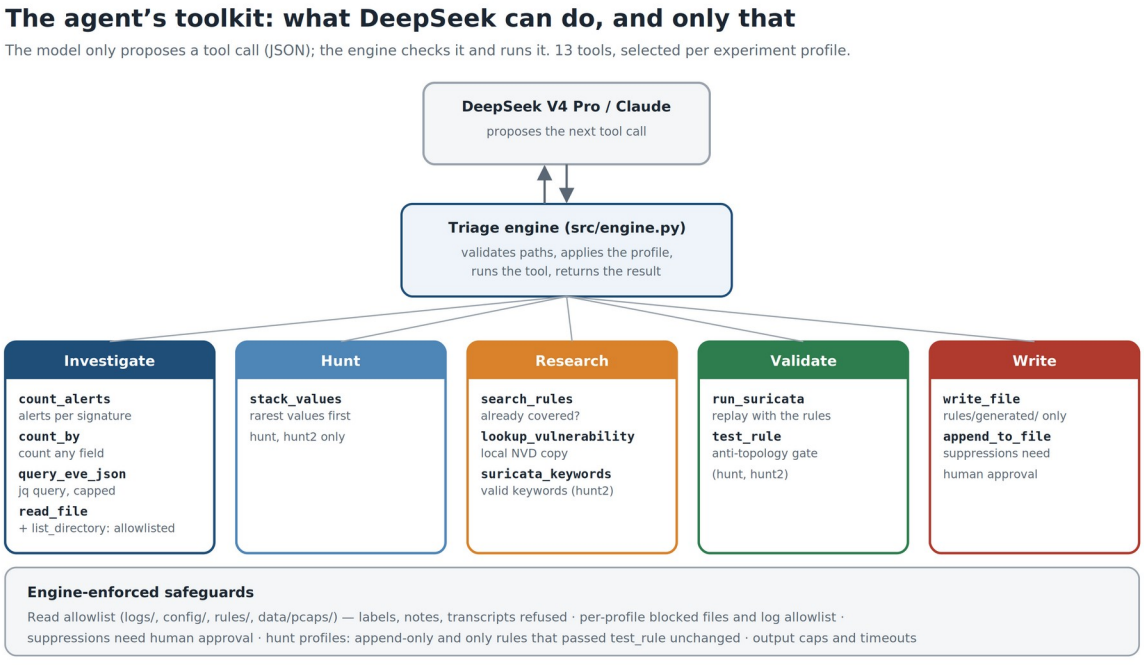


Fig. 3-4 : The agent's toolkit grouped by purpose, with the engine-enforced safeguards

Web Interface

The entire triage process is displayed in real-time in a web interface, with the model's investigative decisions, tool calls, and reasoning being displayed. Multiple browser windows can be used for concurrent triage sessions, allowing multiple datasets or models to be compared. The

evaluation resulted in four concurrent runs of triage being performed without any problems, using a single server instance.

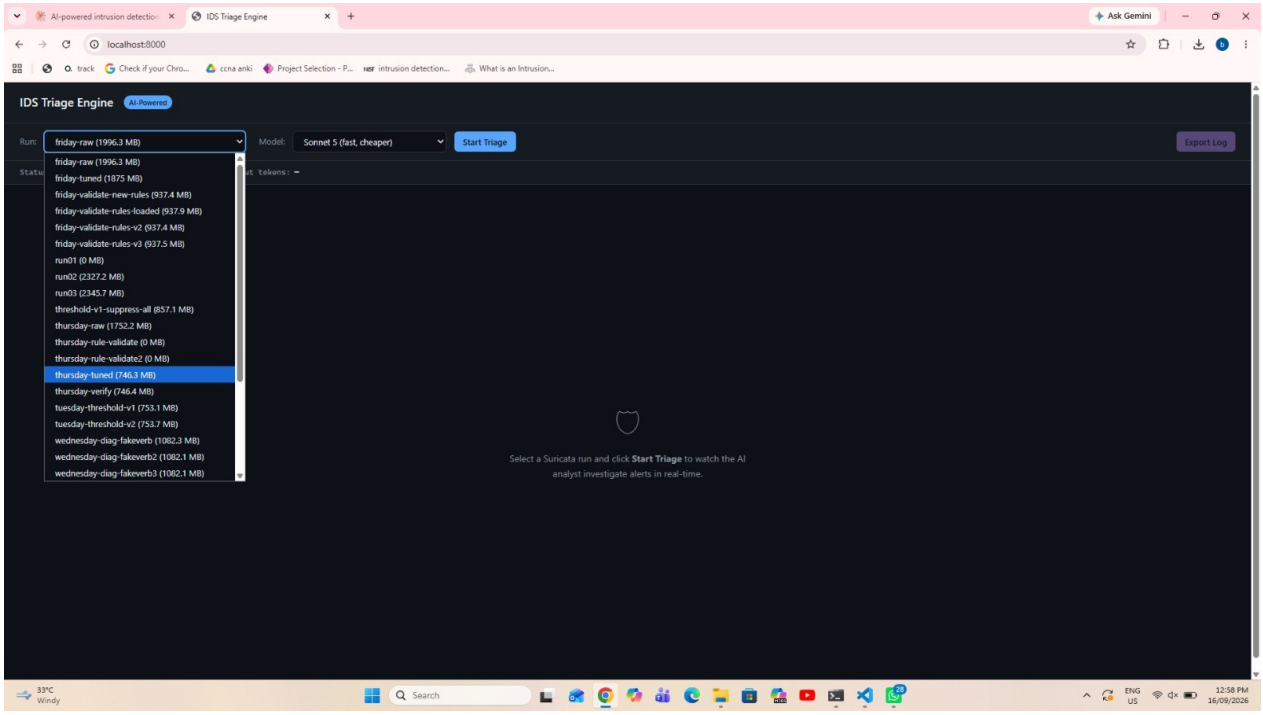


Fig. 3-5 : Web UI screenshot showing model selection dropdown, run selector, and live triage streaming panel

Dataset Configuration and Evaluation Integrity

Dataset abstraction. The first version of the system prompt hard-coded the CIC-IDS2017 network layout, and the model's rule-validation tool always used the CIC Suricata configuration. For the second dataset both were parameterised: each dataset now has its own network-layout statement, Suricata configuration (HOME_NET) and generated-rule file, selected per run, so that rules the model validates are tested under the correct network definition. The triage procedure, tools and model are otherwise unchanged.

Tool-access allowlist. An audit of the triage transcripts (Section V, Threats to Validity) found that earlier runs had listed the ground-truth directory, whose file names reveal the attack types, and that one Tuesday run had read the ground truth from the project's own notes. The tools now open only the directories an analyst at a sensor would use — logs/, config/, rules/ and data/pcaps/ — and refuse labels, project notes, earlier transcripts and results. No UNSW-NB15 run attempted to read the labels.

Alerts-only condition. For Experiment 1 the same engine can run in a mode in which every query is restricted to alert events: flow and application-layer events, protocol statistics and rules written under the full-context condition are unavailable to the model. This isolates the effect of context on triage quality.

Tools and Software

Table 3-2 lists every tool and library the project used, its version and what it was used for. All are open source or open access, except the two hosted model APIs, which the system calls but does not depend on (Section III, Provider Adapter Pattern).

Tool / library (version)	Role in this project
Suricata 8.0.6 (OISF)	The rule-based IDS: offline pcap replay (suricata -r), rule loading and syntax checks (-T), eve.json output, keyword list for suricata_keywords.
suricata-update 1.3.8	Downloads and merges the ET Open ruleset into rules/suricata.rules.
ET Open (Proofpoint)	Community ruleset (52,647 rules); the stock baseline for both datasets.
Snort Community, Stamus lateral, PT Open	Additional open rulesets evaluated for UNSW-NB15 baseline v2; disabled ET and Snort families re-enabled by service.
jq 1.7	Every query the agent runs over eve.json (query_eve_json, count_by, stack_values).
tshark / editcap 4.2.2 (Wireshark)	Checking capture dates and timestamps, slicing captures for fast iteration.
Python 3.12	Triage engine, tools, provider adapters, evaluation and figure scripts (src/).
FastAPI 0.141 + Uvicorn 0.52	Web server that streams the agent's reasoning and tool calls to the browser via Server-Sent Events.
openai 3.7 SDK → DeepSeek V4 Pro / V3	OpenAI-compatible client for DeepSeek, the model used for almost all runs.
anthropic 1.2 SDK → Claude Opus 5	Client for Claude, used for the early CIC-IDS2017 triage.
Local NVD knowledge base (SQLite)	Offline copy of the NVD CVE database (2002–2016) behind lookup_vulnerability; no live web access.
Agent tools (src/tools.py)	count_alerts, count_by, query_eve_json, read_file, write_file, append_to_file, run_suricata, list_directory; per profile: search_rules, lookup_vulnerability, stack_values, test_rule, suricata_keywords.
matplotlib 3.11 + CairoSVG 2.9	All charts (matplotlib) and diagrams (hand-written SVG rendered to PNG) in this report.
python-docx 1.2, pptxgenjs 4.0	Building this report and the presentation from the results files.
Ubuntu 24.04 on WSL 2, Git	Development environment and version control (one repository, four contributors).

Table 3-2: Tools and software used

Experiment Profiles

As conditions multiplied, each was defined as a named profile: the dataset, the Suricata configuration, the single rule file the model may write to, the tool set, the prompt, and the files the model may not open (Figure 3-6). The web interface, the command-line runner and the unattended batch runner all select a profile by name, and every transcript records it. A profile used in a scored run is frozen; a change creates a new profile, so that results can always be traced to the exact set-up that

produced them. The web interface gained a profile selector next to the model selector, filtered to the profiles of the selected run's dataset.

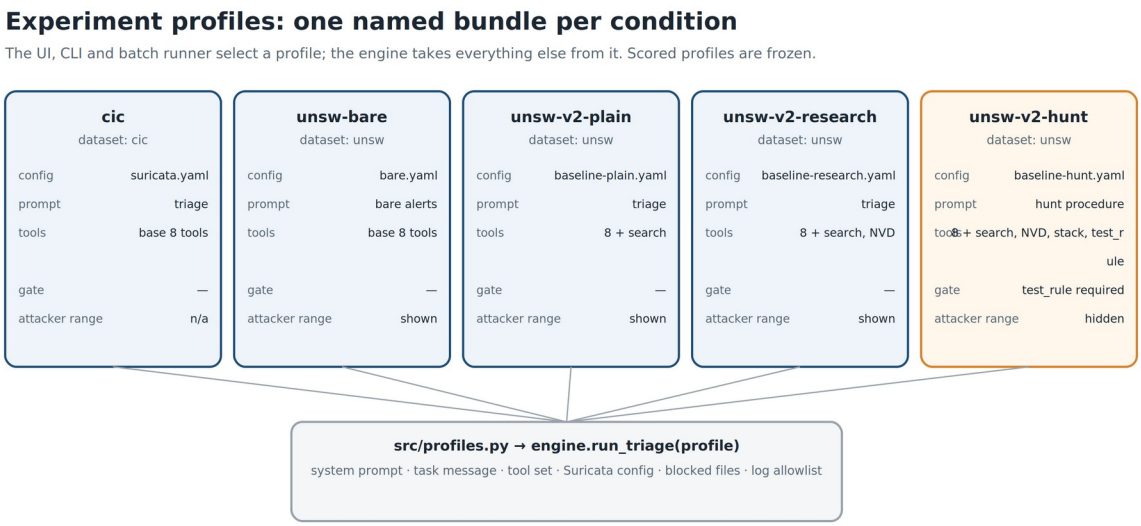


Fig. 3-6 : Experiment profiles — one named, frozen bundle of configuration, prompt and tools per condition (the UNSW v1 profile, which predates profiles, is omitted)

The Hunt Condition: a Detection-Engineering Workflow

Following the supervisor's advice to look at how detection engineers work in practice and replicate it with the agent, a third rule-writing condition was built around the lifecycle described in Section II (Figure 3-7). The triage conditions start from alerts, so the model only ever investigates traffic that some rule already fired on. The hunt procedure starts from the gaps instead: (1) map which protocols carry traffic but few alerts; (2) stack the attacker-controlled content fields of those protocols — URLs, user agents, FTP and SMTP commands, DNS names, SSH client versions, parser anomaly events — rare values first, without pivoting on IP addresses; (3) form a hypothesis, checking whether a loaded rule already covers it and looking up the vulnerability; (4) write the rule with an ADS-style comment stating its goal, blind spots and expected false positives; (5) test it; (6) save it.

Two tools were added. stack_values returns the values of any field rarest first, with the number of distinct values — the existing aggregation tool listed the most common values first and its output cap cut off exactly the long tail a hunter needs. test_rule checks a candidate rule's syntax, replays a capture with only that rule, then replays it again with the rule's addresses and ports widened to any → any, and reports the alerts of each version by direction (Figure 3-8). If the widened rule fires between internal hosts, its precision came from the source address rather than from the

behaviour it claims to detect, and the verdict is FAIL; a rule that raises no alerts also fails. Each test takes about 20 seconds on a 2 GB capture.

Traffic between internal hosts is treated as known-good on the basis of the dataset's published testbed description, in which two of the three servers generate normal traffic and one generates the attacks (Moustafa & Slay 2015; Fraunhofer FKIE 2026). This is documented topology, not per-event ground truth, and the tool reports counts by host role only. The engine enforces the gate: in this profile it refuses any write except an append to the profile's rule file, and refuses any rule that has not passed test_rule in exactly the form being saved. The hunt prompt also omits the attacker range that the triage prompts state, because a real detection engineer does not know it; the hunt condition therefore differs from the triage conditions in both procedure and context.

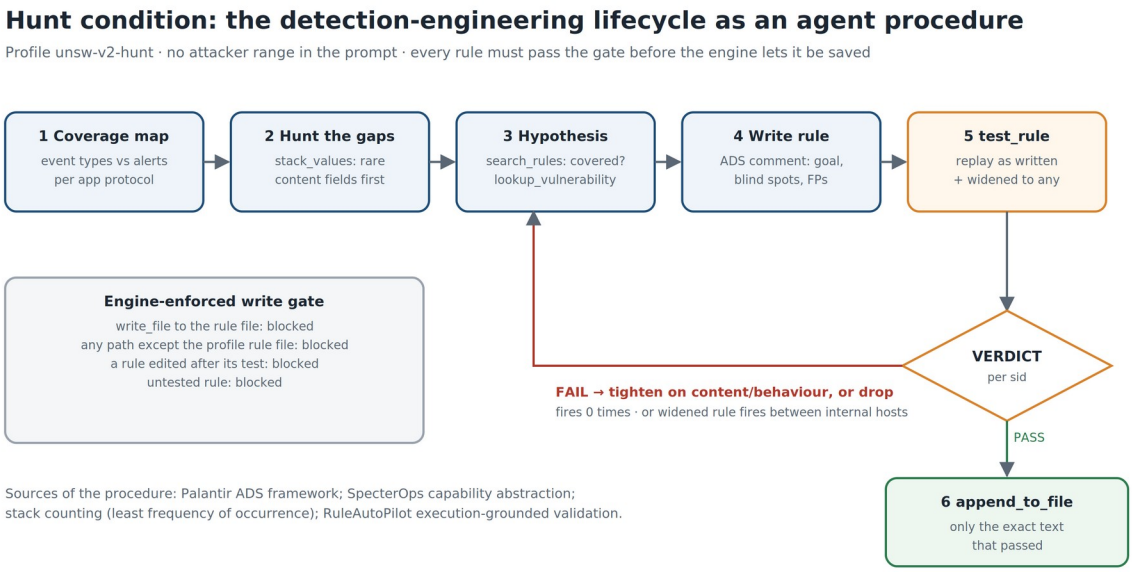


Fig. 3-7 : The hunt condition — the detection-engineering lifecycle as the agent's procedure, with an engine-enforced test gate

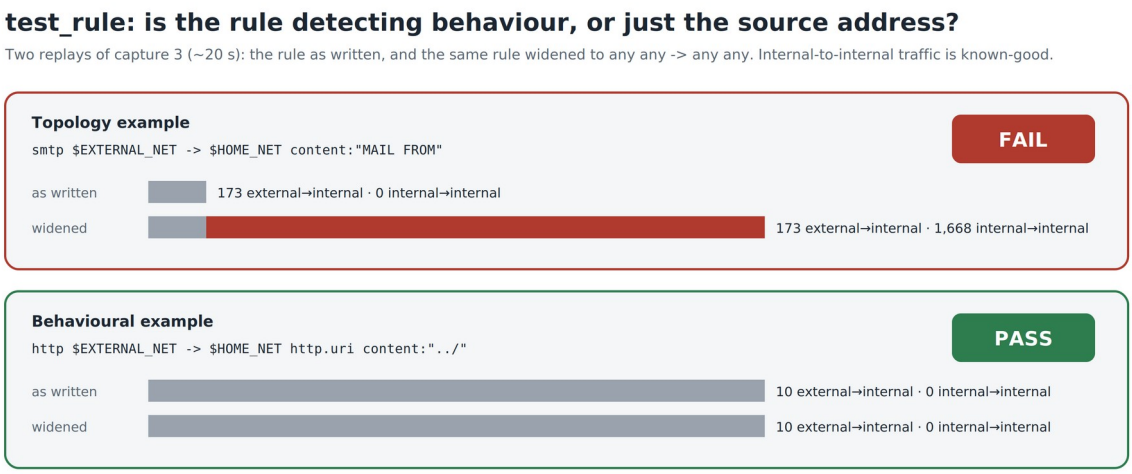


Fig. 3-8 : test_rule — a topology rule fails when widened to any → any; a behavioural rule passes (capture 3)

A fourth condition, hunt2 (profile unsw-v2-hunt2), is identical to the hunt condition except for one further tool. suricata_keywords lists the rule keywords the installed Suricata 8.0.6 supports, searched by name or description, and for an exact name returns its description and documentation link. It was added after the hunt transcripts showed the model guessing keywords that do not exist in this version (for example smb.filename) and falling back to raw-byte regular expressions. Hunt2 writes to its own rule file with its own SID range and cannot open the hunt condition's rules or logs, so the difference between the two conditions measures the tool.

Changes from Original Project Plan

Several iterations were carried out in the development of this understanding, leading to design improvements:

- Replacement of pass rules with alert-only rules. The original design called for Suricata pass rules to be used for allowlisting known-good traffic. It was found that if a pass match occurs on a TCP connection, the entire flow passes, and Suricata will no longer scan it. If a normal pass match passes an attacker's packets, the rest of the connection will be scanned by nothing. All Rules are now alert only.
- Replacement of custom normality rules by ET Open plus threshold suppression. The original idea was to create detection rules for each service. Releases a more defensible, wide-ranging detection surface at the outset, starting from a mature community ruleset and tuning through suppressions, emulating a true SOC workflow.
- The scope of the work was extended beyond rule generation to complete triage automation. The system automates the entire analyst workflow, from investigation to correlation, decision making, and rule authoring, as opposed to building rules from individual alerts.
- CIC-IDS2017 was finalized to set as a test environment, as its Ground truth is reproducible and labeled for evaluation, enabling systematic comparisons that cannot be demonstrated by example alone.
- UNSW-NB15 was added as a second evaluation dataset at the supervisor's request (Week 9 meeting), to test whether the approach generalises beyond the dataset on which it was developed.

- The UNSW-NB15 baseline was strengthened before further AI rounds, by enabling rule families that ET Open and Snort ship disabled, selected from the services seen in the traffic, so that the AI is compared with the strongest human baseline available rather than with an untuned ruleset.
- A hunt condition replicating detection-engineering practice was added after the supervisor advised looking at real-world practice, and round results are now credited only for attack events that no other rule detected.

Summary

The method focuses on equipping the generative language model with the tools and step-by-step instructions it needs to perform the entire analyst workflow. The system retains the rule-based detection all the way through, has a safety asymmetry between detection and suppression, and allows for fair model comparison via a provider adapter pattern. The design focuses on the inspectability of the rules, the decisions made during each investigation, and the human-readable results of all rules; the steps taken during each investigation are recorded, and each decision made during an investigation can be traced back to a specific rule.

IV. Execution and Results

This part explains the implementation of the triage engine for the four attack days of the CIC-IDS2017 dataset and the results achieved. These attack days were run in at least two passes: a tuned pass with the Monday baseline suppressions applied, and a raw pass with no suppressions applied to the data, for comparison of the effectiveness of the triage under different noise conditions. A second dataset, UNSW-NB15, was then evaluated in raw mode only (see Cross-Dataset Evaluation: UNSW-NB15).

Evaluation Dataset

The Canadian Institute for Cybersecurity (CIC-IDS2017) at the University of New Brunswick has generated the CIC-IDS2017 dataset (Sharafaldin, Habibi Lashkari & Ghorbani 2018). It is a set comprising 5 days of network traffic captured on a dedicated testbed representing a realistic enterprise network, along with normal traffic and a range of simulated attacks.

Testbed topology: The victim network has an address of 192.168.10.0/24, and includes the following: a web server running non-standard port TLS services at 192.168.10.51 (Ubuntu); a Windows domain controller at 192.168.10.3 (domain Testbed1.ca); a web server running non-standard port TLS services at 192.168.10.50 (Ubuntu, running DVWA

(Damn Vulnerable Web Application)); and several workstations at .8, .12, .25, and others. The external attacker is from 172.16.0.1, and a command and control server is 205.174.165.73. Throughout the results, this topology will be referred to as such.

Capture structure: The first day (Monday, Monday-WorkingHours.pcap) consists of only benign traffic. It is used as the database for baseline sensor tuning, which resulted in 529,918 flows with a ground truth label BENIGN. On the other four days, the tests were run with different attack families for each stage, in addition to the normal background traffic.

Ground truth labels: CICFlowMeter creates an 85-column CSV file for each capture it captures, with per-flow features. Each flow is labeled BENIGN or one of the attack categories in the Label column (column 85). These labels serve as ground truth for measuring detection coverage. The structure of the data is summarised in Table 4-1.

Weekday	Dataset Filename	Attack Categories	Labeled Attack Flows
Monday	`Monday-WorkingHours`	None (benign baseline)	0
Tuesday	`Tuesday-WorkingHours`	FTP-Patator, SSH-Patator	7,938 + 5,897
Wednesday	`Wednesday-workingHours`	DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest, Heartbleed	231,073 + 10,293 + 5,796 + 5,499 + 11
Thursday	`Thursday-WorkingHours`	Web Attack Brute Force, Web Attack XSS, Web Attack SQL Injection, Infiltration	1,507 + 652 + 21 + 36
Friday	`Friday-WorkingHours`	DDoS, PortScan, Bot (ARES)	128,027 + 158,930 + 1,966

Table 4-1: CIC-IDS2017 dataset structure ,captures, attack categories, and ground-truth flow counts.

Why this dataset: The choice of CIC-IDS2017 is motivated by the following factors:

- It provides labeled ground truth at the flow level.

- it contains a wide variety of attack types such as: brute force, denial of service, web application attacks, infiltration, DDoS, port scanning, and botnet.
- The attacks are accompanied by realistic background traffic.
- It is publicly available for reproducibility, but it is accepted that it has limitations: The traffic is nine years old, the attack tools are too old, and the goal isn't to cover up to the latest threat; it is supposed to prove out the triage methodology.

Credential Brute Force

This capture (Tuesday-WorkingHours in the dataset) yielded 2,404 alerts after baseline suppression in two signatures. The ground truth labels show 7,938 FTP-Patator and 5,897 SSH-Patator attack flows.

Both Claude Opus 5 and DeepSeek V3 recognized:

- Attacker 172.16.0.1 – 11891 failed logins (response code 530) and 1 successful login (response code 230) with a password of 1234 to victim 192.168.10.50. There was no data exfiltration after login - only a SYST command and session timeout.
- Brute force SSH via Paramiko 2.0.0 toolkit: ~2980 flows to SSH port 22 - none of them were successful attempts to log in via SSH.

Claude authored two rules for the campaign level (SID 9500001–9500002). DeepSeek created five rules with a stateful, two-stage detection chain powered by Suricata's hostbits facility (SID 9000001 – 9000005): a rule is triggered when an attacker's IP is detected for repeated failures, then another rule is triggered for a successful login to that IP. Ground-truth replay (Section V) later showed that four of the five DeepSeek rules, including both stages of the hostbits chain, never fired on the Tuesday capture; only SID 9000004 (the Paramiko client fingerprint) fired, matching 574 SSH-Patator flows with no false positives.

Fig. 4-1: Web UI screenshot showing the triage engine during a live investigation, with tool calls and model reasoning visible in the streaming panel (illustrative run captured for this figure, not part of the evaluation set)

Denial of Service and Heartbleed Exploitation

This capture was made on two passes (Wednesday-workingHours in the dataset). DeepSeek V3 was used for the tuned pass, and DeepSeek V4 Pro was used for the raw pass.

Tuned pass (381 alerts, DeepSeek V3 : The model detected Slowloris/Slowhttptest denial-of-service traffic, 167,552 HTTP requests were sent between 172.16.0.1 and 192.168.10.50, of which 867 were FAKEVERB, and 1,054 were requests that contained the slowhttptest toolkit referer. Heartbleed probing (CVE-2014-0160) was also identified on a non-standard TLS port (444) targeting 192.168.10.51. V3 wrote three detection rules (SID 3000001-3000003): a fingerprint for the slowhttptest toolkit, sustained referer bursts, and FAKEVERB method requests.

Raw pass (2,310,105 alerts, DeepSeek V4 Pro): eventhough 99.97% of alerts were engine noise, the model was able to identify the same attacks, and even an attack type not present in the stock ET Open ruleset, a Hulk-like HTTP GET cache-busting flood (149,703 requests). A new rule (SID 3000004) has been written and validated at 95 hits. V4 Pro also noticed that V3 fired its earlier rules in the raw data – SID 3000001 (Slowloris toolkit fingerprint) was fired 187 times as opposed to 3 hits for the stock ET rule. V4 Pro found a bug in SID 3000003 (FAKEVERB detection) and diagnosed the cause: the HTTP parser in Suricata does not expose non-standard HTTP methods to the http. method detection buffer, so V4 Pro rewrote the rule using raw TCP payload matching at depth:8, after which it fired 463 times. V4 Pro also removed SID 3000001 and

3000002 from the rules file, stating that these were stock ET Open rules - this was false (see Section V, Model Errors).

The model also conducted a forensic analysis on the Heartbleed exploit and found that the attacker sent 1.9 MB of crafted heartbeat requests and received 81.5 MB of memory from the server in return, compared to the normal size of a TLS handshake, which is 350-850 bytes. V4 Pro created 3 rules for Heartbleed (SID 7600101 - 7600103) for Heartbleed itself, Heartbleed reconnaissance burst, and probing of non-standard TLS ports. Of these, SID 7600101 can never fire in Suricata 8.0.6 because the TLS event it relies on is not registered, and the Heartbleed rule file was not loaded in any Wednesday replay. SIDs 7600102–7600103 were later found to fire on real exploit traffic in UNSW-NB15.

Web Application Attacks and Network Infiltration

This capture (Thursday-WorkingHours in the dataset) includes Web Application Attacks and Network Infiltration. Triage was done with tuned (25,406 alerts) and raw (1,346,596 alerts) configurations.

Web attacks (172.16.0.1 → 192.168.10.50): The model found three attack phases to the web app of DVWA(Damn Vulnerable Web Application):

- **HTTP login brute force:** 1,826 POST requests to /dv/login.php at 47-48 requests per minute with 1,820 failed attempts and 6 successful login attempts. This was a very significant coverage gap, with no stock ET ruleset firing any brute-force signature for this attack. Ground-truth replay confirmed the gap: no stock alert matched any of the 1,507 labelled brute-force flows, whereas the AI rule SID 7600001 matched 8 of them with no false positives.
- **Reflected cross-site scripting:** This traffic was triaged by the Suricata SID 2009714 with 891 unique alerts, while the Suricata triage model reported more than 1801 occurrences of script probes to the same endpoint in its documentation. The number of alerts and the number of HTTP requests do not match; the flow-level evaluation in Section V measures detection against the 652 labelled XSS flows directly. All successful (HTTP 200 responses) contained <script> payloads executed. These were correctly detected by Stock ET SID 2009714.
- **Union-based SQL injection:** SQL Injection using the UNION method: 15 requests to /dv/vulnerabilities/sqli/ such as union select database(), user(), union selects null, table_name from

information_schema.tables, and union select user, password from users. All HTTP 200 responses returned with data (extraction was successful). Stock ET rules 2010963, 2017808, and 2016672 detected these.

Infiltration (192.168.10.8 compromised): The model followed a full post-compromise kill chain from the internal host (192.168.10.8):

- **Reverse shell:** 5 outbound TCP connections to command and control server 205.174.165.73:444 with 15.2 MB of interactive Windows cmd.exe session information.
- **Lateral movement:** Sweep SMB NEGOTIATE_PROTOCOL to 6 internal hosts (.9, .12, .16, .17, .19, .50), followed by reading of SYSVOL GPO and DCE/RPC SAMR/LSARPC domain enumeration on the domain controller.
- Data staging (retracted): the model reported 44 hex-named .txt uploads from .8 using the Tuesday credentials (iscxtap/1234). Ground-truth replay does not support this. The rule written for it (SID 7600004) fired on uploads from every internal workstation on both Tuesday and Thursday, and every label-matched hit falls on flows labelled BENIGN. The uploads indicate routine simulated user activity; the account is most likely the testbed's ordinary FTP account, whose weak password the Tuesday attack happened to guess.
- **Internal scanning**: Broad SYN sweep of .8 targeting .5, .14, .15 with around 28,000 flows at peak rates of 2,000 SYNs per minute. The labels mark these flows BENIGN. Because CIC-IDS2017 is known to leave some performed attacks unlabelled (Lanvin et al. 2023), this is treated as a disputed label rather than a false positive.

Raw pass additional finding: The second reverse shell victim (192.168.10.25) was also discovered communicating with this same C2 server, and an additional 13.6 MB was downloaded. No alert was generated for this host by Stock ET; it was only detected by flow-level analysis of the raw data.

Six detection rules were written (SID 7600001-7600006) for Web Login Brute Force, SMB Lateral Sweep, Reverse Shell C2, FTP Data Staging, SYSVOL GPO Recon, and DCE/RPC Enumeration. Another rule (SID 7600007) was created from the raw pass to identify the internal sweep of SYN. On replay against the Thursday capture, SID 7600001 matched 8 labelled brute-force flows, SID 7600003 matched 2 Infiltration flows (and 2

benign), SID 7600004 fired only on benign traffic, and SIDs 7600002, 7600005 and 7600006 fail to parse in Suricata 8.0.6 (unknown keyword smb.command; malformed dcerpc.iface), so they never loaded. The model's own validation of these rules had replayed a small test capture, not the Thursday traffic.

Distributed Denial of Service, Port Scanning, and Botnet

This capture (Friday-WorkingHours in the dataset) shows a lot of DDoS, port scanning, and botnet activity. Triage was performed with both the tuned (263 alerts) and raw (1,412,449) configurations. The tuned log used for triage contained two appended Suricata runs, so the model analysed 525 alert events — every alert twice.

Port scan (172.16.0.1 → 192.168.10.50): A wide range of TCP SYN sweeps across some 1,000 ports and more than 160,000 inbound flows, including banner grabbing of services found (FTP, SMB, SSH). Only the fixed subset of known ports - MySQL, PostgreSQL, Oracle, and VNC - was detected by Stock ET SCAN rules.

SSH brute force: 526 flow records to port 22 and 62 SSH banner exchanges from the known attacker host (172.16.0.1). SID 9500002, written by Claude for Tuesday, fired on this traffic. The label join (Section V) shows that all 79 of its label-matched Friday alerts fall on flows labelled PortScan, so this is the SSH phase of the port scan rather than a separate brute-force campaign.

HTTP GET flood : Approximately 160 requests per second of 189,188 brief GET requests that did not include a User-Agent field. Every 189,128 responses came back with HTTP 200, which meant it was a legitimate-looking request, and the web server was being overrun with traffic. This 189,000-request campaign only received 40 fires from the stock ET rule 2019348. The AI rule written for this campaign (SID 9500003) was never loaded in any replay because its rule file was not listed in the Suricata configuration, so its effect could not be measured; the equivalent campaign rules SID 9600001 and its copy 9600003 matched 49 and 193 DDoS-labelled flows respectively.

Botnet (ARES). In a later validation run on 21 September 2026, prepared for the Week 9 supervisor demonstration, DeepSeek V4 Pro found the ARES botnet that the interim report had listed as undetected: five internal hosts (.5, .8, .9, .14, .15) polling the C2 server 205.174.165.73:8080 with 666 GET /api/pop?botid=… requests and 70 POST /api/upload or /api/report requests, all with the python-requests user agent. No stock or custom alert fired on this traffic. The model wrote three rules (SID 9700001–9700003) that each fired five times on replay — once per

infected host — and all 15 alerts match Bot-labelled flows. The transcript of this session was not exported, so the model's reasoning cannot be audited; the rules, their commentary and the validation logs are the evidence.

Summary

Across the four attack days the model identified 13 of the 14 labelled attack categories (DoS slowloris and DoS Slowhttptest were reported as a single campaign, and only Slowhttptest flows were matched by the resulting rules); only DoS GoldenEye (Wednesday, 10,293 labelled flows) was never named in a triage report. Bot/ARES, reported as undetected in the interim report, was found in the 21 September run. Across all CIC-IDS2017 sessions the model wrote 32 rule definitions (27 distinct detections once copies and superseded drafts are removed); Section V audits each of them. These fill in several coverage gaps in the stock ET Open ruleset. These results showed that, although the cost overheads are significantly reduced by baseline suppression, it is possible that opportunities to detect are not visible in raw passes. The results of the execution are summarised in Table 4-2.

Capture	Model(s)	Tuned Alerts	Raw Alerts	Attack Categories Found	Rule definitions	Key Gaps Closed
Tuesday	Claude Opus 5, DeepSeek V3	2,404	1,842,712 (later replay)	FTP brute force, SSH brute force	7 (SID 9000001–9000005, 9500001–9500002)	Stateful brute force chain
Wednesday	DeepSeek V3, V4 Pro	381	2,310,105	DoS Hulk, Slowloris, Slowhttptest, Heartbleed	5 (SID 3000003–3000004, 7600101–7600103); +2 deleted	FAKEVERB DoS, Hulk flood, Heartbleed

Thursday	DeepSeek V4 Pro	25,406	1,346,596	HTTP brute force, XSS, SQLi, Infiltration (reverse shell, lateral movement)	7 (SID 7600001–7600007)	Web login brute force, 2nd reverse shell victim
Friday	DeepSeek V4 Pro	263	1,412,449	DDoS, PortScan, Bot (ARES)	3 (SID 9500003, 9600001–9600002); 11 incl. copies and ARES rules (App. D)	HTTP GET flood campaign, broad SYN sweep, ARES botnet C2

Table 4-2: Execution summary — alert volumes, attack categories identified and AI-authored rule definitions per capture

Figure 4-2 compares alert volumes. Raw CIC-IDS2017 captures produce between 1.3 and 2.3 million alerts, most of them Suricata engine events such as invalid-checksum alerts; baseline suppression reduces this by three to four orders of magnitude. UNSW-NB15 raw captures produce fewer than a thousand alerts each, because the checksum artefact is specific to the CIC capture.

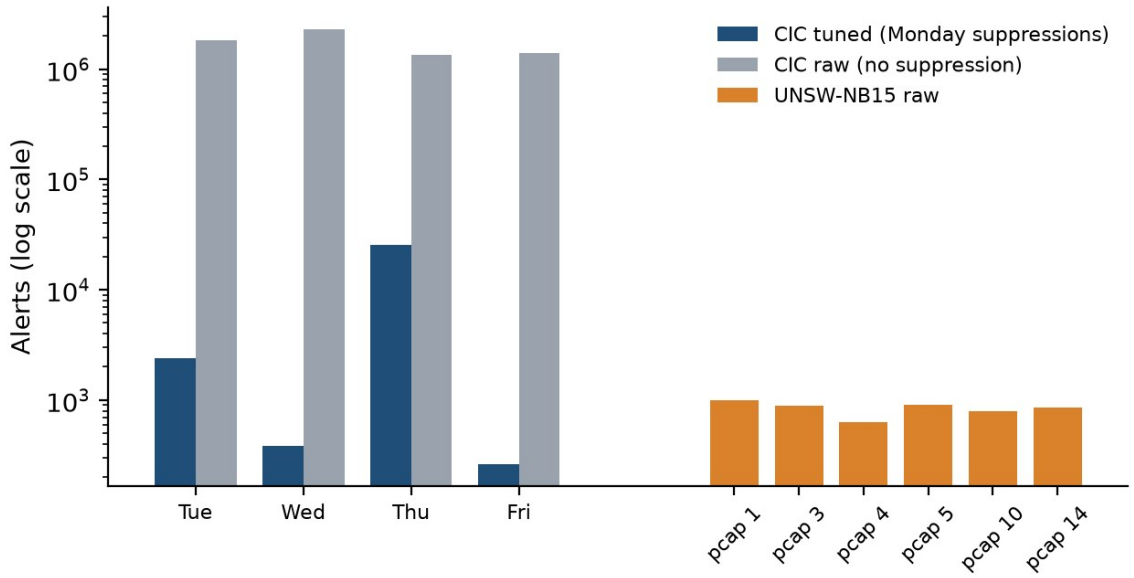


Fig. 4-2 : Alert volume per capture — CIC-IDS2017 tuned and raw passes, UNSW-NB15 raw (log scale)

Cross-Dataset Evaluation: UNSW-NB15

At the supervisor's request (Week 9), the triage engine was evaluated on UNSW-NB15 (Moustafa & Slay 2015), generated with the IXIA PerfectStorm tool in the UNSW Canberra Cyber Range Lab as a mix of real normal activity and synthetic attacks in nine families: Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode and Worms. Its ground truth (NUSW-NB15_GT.csv) lists every attack event with start and end time, endpoints, protocol and category. Six 2 GB capture files from the 17 February 2015 capture were used (files 1, 3, 4, 5, 10 and 14; a seventh, file 27, was truncated during download and contained no ground-truth events). Together they contain 31,432 labelled attack events, 14,935 of them TCP or UDP, covering all nine categories.

The network layout was derived from the ground truth and the traffic itself: every attack originates in 175.45.176.0/24 and targets 149.171.126.0/24, while 59.166.0.0/24 generates normal client traffic, so HOME_NET was set to the server and client ranges in a separate configuration. UNSW-NB15 has no benign-only day, so no Monday-style baseline could be built and all UNSW runs are raw. Stock ET Open was loaded together with the rules written on CIC-IDS2017, to test whether they transfer. Apart from the network-layout statement, the prompt, tools and model (DeepSeek V4 Pro) were unchanged. Table 4-3 summarises the six runs.

Capture	Raw alerts	From attacker range	Turns	Duration	Rules written	Model validated its rules
unsw-1	997	79%	75	12 min	2	Yes
unsw-3	893	85%	44	10 min	3	Yes
unsw-4	625	77%	44	33 min	2	Yes
unsw-5	905	87%	43	111 min	5	Yes
unsw-10	799	77%	62	9 min	3	No (replayed by the team)
unsw-14	854	83%	50	41 min	3	Yes

Table 4-3: UNSW-NB15 triage runs (DeepSeek V4 Pro, raw captures)

The alert stream is very different from CIC-IDS2017: 625–997 alerts per capture, 77–87% of them involving the attacker range. It is also sparse relative to the attacks: capture 3 alone contains 18,506 labelled attack events but produced 893 alerts, so the stock ruleset sees only a small part of the attack activity. Run duration varied from 9 to 111 minutes for a

similar number of turns, which reflects API response times rather than task difficulty.

Figure 4-3 shows which ground-truth categories each run's findings correspond to, obtained by looking up the hosts and ports cited in each finding in the ground truth. Across the six runs the model identified eight of the nine categories; each run found four to seven. Some identifications were exact: the unsw-3 report names SQL Slammer worm propagation to 149.171.126.17:1434/UDP, which is the single Worms event in that capture, and outbound cmd.exe shell banners reported in unsw-1 and unsw-5 match labelled Backdoor events. Analysis (port scans, spam and HTML penetration) was never identified.

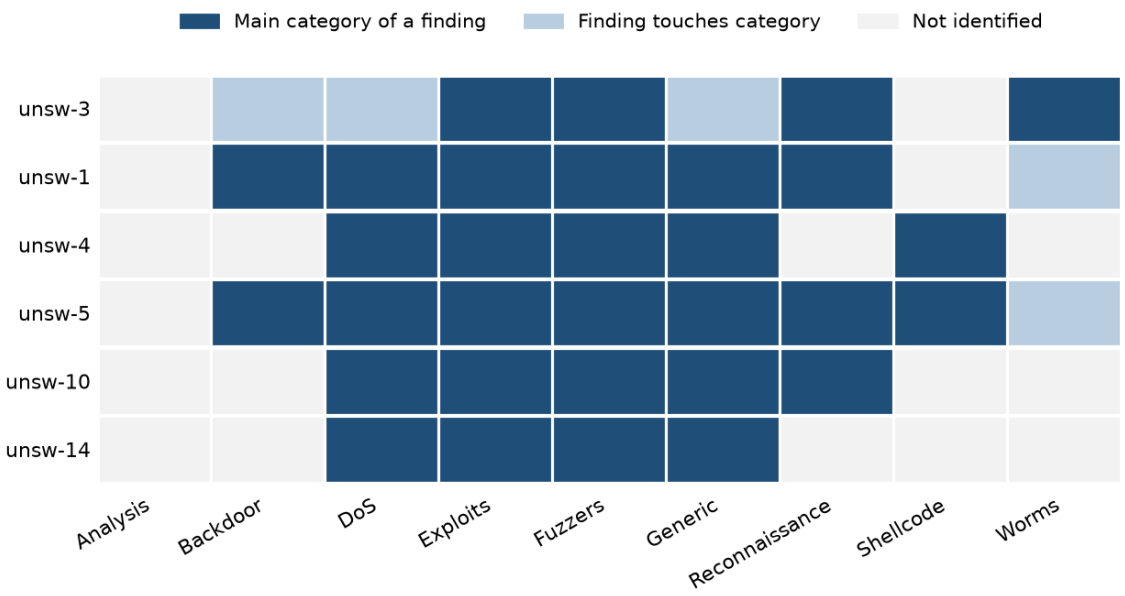


Fig. 4-3 : Ground-truth categories identified by each UNSW-NB15 triage run

The six runs wrote 18 rules. All of them load and fire on replay, because the UNSW configuration listed the model's rule file from the outset (contrast the CIC rule audit in Section V). One run, unsw-10, skipped its own validation step; the team replayed its three rules, which fired 2, 4 and 1 times, all on attacker traffic. Four rules duplicate existing ones: two re-author the CIC Heartbleed rules with a new message, and two repeat earlier UNSW rules even though the model had read the rule file first. Rules transfer across captures: with every rule loaded, 13 of the 15 rules written on other captures fired on capture 14.

The CIC-authored Heartbleed rules SID 7600102–7600103 fired on all six captures, only on attacker traffic, and matched 495 ground-truth attack events, mostly in the Generic category. Five of the six runs judged these alerts true positives. The unsw-3 run judged them stale false positives — the traffic was a large upload, the opposite of a Heartbleed leak — and

proposed suppressing both rules. The ground truth labels that exact flow (175.45.176.0 → 149.171.126.18:2444) as a Cisco Unified Communications Manager heap-overflow exploit, which is precisely why it was a large upload. Because suppressions require approval, the proposal was not applied. Had it been applied automatically, the model would have blinded the sensor to a real exploit — direct evidence for the safety asymmetry described in Section III. A later rule-by-rule classification (Section V, Attribution and the Topology Trap) qualifies the first part of this paragraph: both rules fire on any outside host that opens five TLS connections within five minutes. In UNSW-NB15 every outside host is an attacker, so their matches show where the attackers are, not that the CIC rules recognise an attack technique; they are not evidence that CIC-authored rules transfer to UNSW-NB15.

Challenges of UNSW-NB15

UNSW-NB15 differs from CIC-IDS2017 in ways that shaped every later design decision. Table 4-4 collects the challenges met, what each would have done to the results if ignored, and how the project handled it.

Challenge	Effect if ignored	How it was handled
No benign-only day	No Monday-style tuning or suppressions possible	All UNSW runs are raw; the baseline was strengthened by selecting rule families by service instead
Every external host is an attacker	Rules for ordinary activity from outside ('topology rules') score perfectly but would fail on a real network	Every AI rule classified by its text; genuine gain counts signature and behavioural rules only; test_rule gate in the hunt profiles
No internal POP3, SIP or SMB traffic	The gate cannot test rules on those protocols against known-good traffic	Stated as a limitation; benign-only hold-out captures used as the only check
Generator varies every attempt (IXIA PerfectStorm)	Rules for one exploit instance fire once and never again	Instance and technique rules distinguished; transfer measured on captures no rule was written on
Low signature coverage	ET Open as shipped detects 9.2% of TCP/UDP attack events	Baseline strengthened to 25.7%; detectability analysis shows most missed events carry data
Event-level ground truth	Alerts cannot be joined by flow ID	Join on endpoints, ports, protocol and time (±2 s); headline on TCP/UDP events only
22 January ground truth covers 11:00–14:00 UTC only	Hold-out captures outside the window contain no labelled attacks	Captures 25 and 50 kept as a false-alarm check; captures 3 and 6 added, chosen by time only
Both days number files from 1; Wireshark shows local time	17 February captures appear as 18 February; easy to mix up the days	Every capture verified by its first-packet timestamp; hold-out kept in its own folder
Truncated and large captures	Capture 27 truncated; 1–2 GB files; stalled downloads	Capture 27 excluded; downloads verified and re-run
10 Snort rules reference an undefined variable	They fail to load in every run	Identical across all configurations, so comparisons are unaffected
Replays are not repeatable	Comparing two replays counts timing noise as gain (+82 vs +6	Gains attributed within a single replay

	for the hunt round)	
Data sent without the TCP PSH flag	Payload cannot be inferred from flags	Content-detectable estimated from byte counts
Suricata normalises URLs	Encoded traversal rules on http.uri match nothing	The agent learned through the gate to use http.uri.raw

Table 4-4: Challenges of UNSW-NB15 and how they were handled

Strengthening the UNSW-NB15 Baseline

The comparisons so far set the AI rules against ET Open as shipped. A detection engineer's first step with a sensor that misses most attacks would be to tune the rules already available, so the baseline was strengthened before any further AI rounds. Four further open rulesets were evaluated (Snort Community, Stamus lateral-movement, PT Open and an Nmap ruleset), and the rule families that ET Open and Snort ship disabled were re-enabled, selected by the services seen under attack in the eve.json traffic profile — never from the labels. Table 4-5 and Figure 4-4 show each step on the six development captures.

Configuration	TCP/UDP events detected	Detection	Alerts
ET Open as shipped	1,381 of 14,935	9.2%	4,475
+ Snort Community	1,398 of 14,935	9.4%	4,521
+ Stamus lateral	1,381 of 14,935	9.2%	4,486
+ PT Open	1,388 of 14,935	9.3%	4,482
+ Nmap scans	1,383 of 14,935	9.3%	4,507
+ all extra rulesets	1,407 of 14,935	9.4%	4,571
+ disabled ET families (selected)	2,461 of 14,935	16.5%	11,108
+ disabled Snort families (selected)	3,048 of 14,935	20.4%	24,193
Baseline v2: all selected families	3,840 of 14,935	25.7%	30,730

Table 4-5: Strengthening the UNSW-NB15 baseline (six development captures)

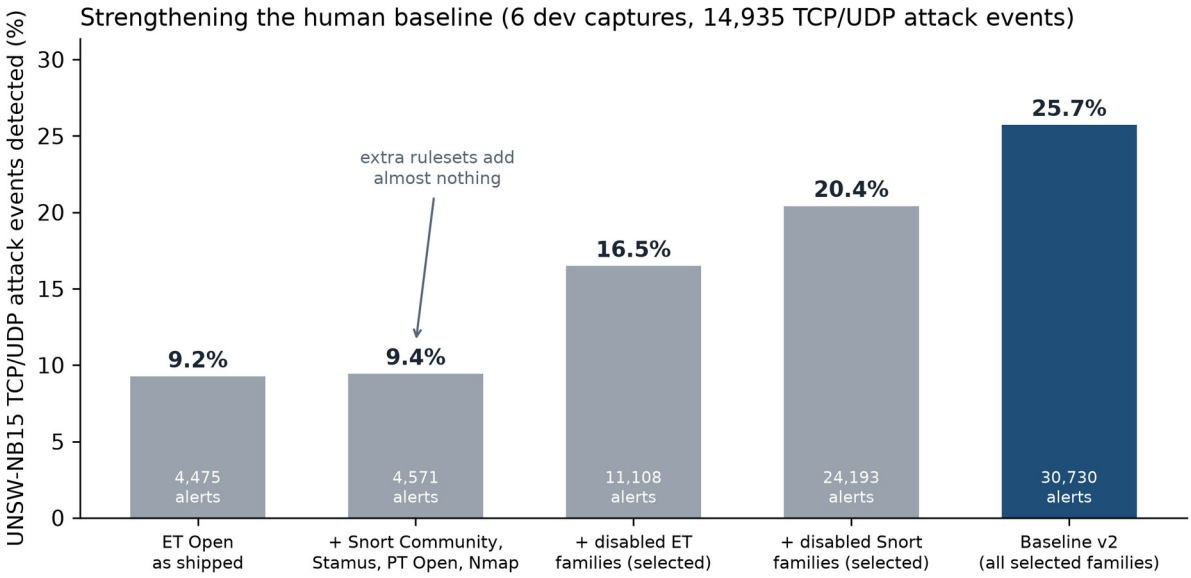


Fig. 4-4 : UNSW-NB15 TCP/UDP detection as rulesets and disabled rule families are added

Additional rulesets add almost nothing; re-enabling the selected disabled families nearly triples detection, from 9.2% to 25.7%. A control run enabled every disabled rule: detection rose to 31.2%, but alerts on hosts with no attacker involved rose from 809 to 37,405 (Figure 4-5) — the selection gives up a few points of detection for 46 times less noise. The Nmap ruleset was dropped because it degraded precision on CIC-IDS2017 without improving UNSW-NB15. The result, baseline v2, is frozen and contains human-written rules only. Replays are not perfectly repeatable — Suricata's thresholds depend on packet timing — so the same configuration scored 24.9% in the later replay used for the AI rounds.

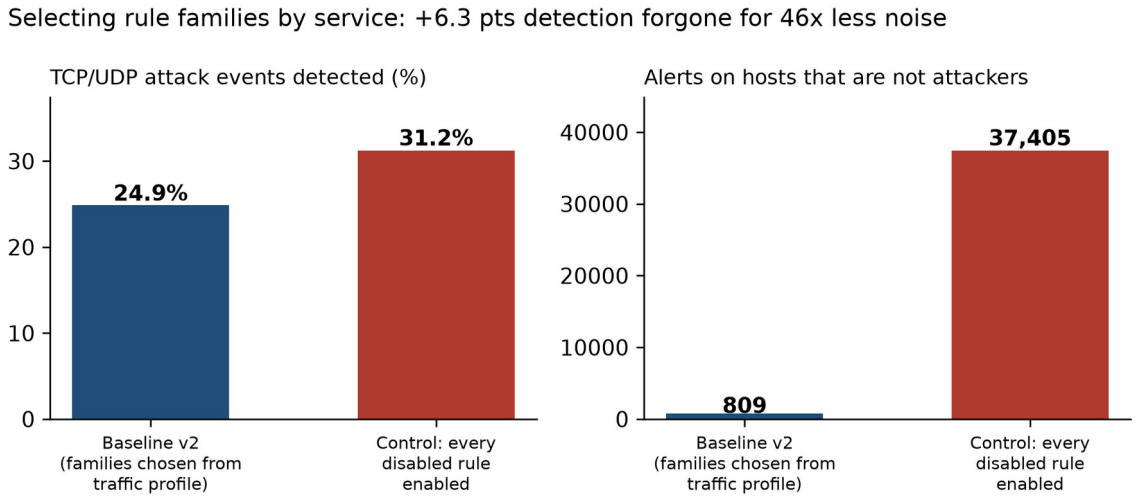


Fig. 4-5 : Selected families versus enabling every disabled rule — detection and alerts on non-attacker hosts

Even the strengthened baseline misses most attack events in every category (Figure 4-6): Exploits 29.0%, Fuzzers 32.4%, Reconnaissance 25.2%, and Generic — 2,677 events — only 7.3%. Most missed events are

one-off attempts whose content varies from attempt to attempt, which matters for what any rule author, human or AI, can add.

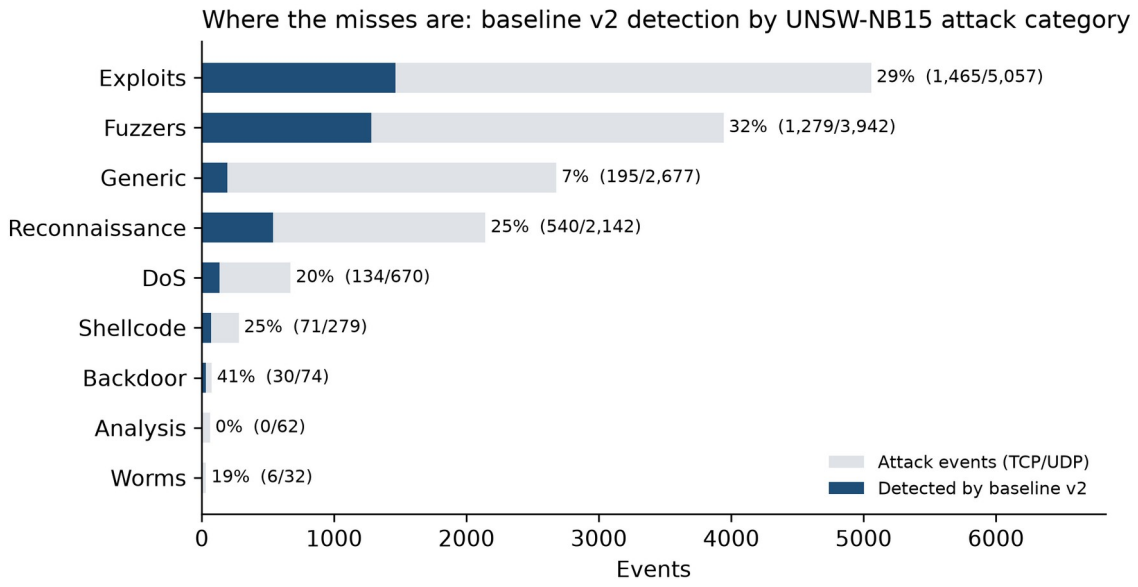


Fig. 4-6 : Baseline v2 detection by UNSW-NB15 attack category

AI Rounds on the Strengthened Baseline (Experiment 3)

Experiment 3 asks whether the agent can add detection to baseline v2. Three conditions were run with DeepSeek V4 Pro, each with its own profile, configuration and rule file so that no condition could see another's rules: plain triage with the search_rules tool (does a loaded rule already cover this?); research, which adds the local NVD vulnerability lookup; and hunt, the detection-engineering workflow of Section III. In each round the model wrote rules on captures 3, 1 and 10 in turn, every round's rules were loaded for the next, and all six development captures were then replayed and scored. Gains are credited only for TCP/UDP attack events matched by an AI-rule alert and by no other alert in the same replay (Section V explains why). Table 4-6 summarises the rounds.

Round	Rules (fired)	AI-rule alerts	Matching an attack	On non-attacker hosts	Attributable gain	Genuine gain
Plain R1	16 (5)	1,034	59.5%	0	+354 (+2.37)	+0 (+0.00)
Plain R2	16 (16)	4,265	82.4%	0	+1,689 (+11.31)	+12 (+0.08)
Research R1	12 (6)	991	87.2%	0	+77 (+0.52)	+18 (+0.12)
Research R2	12 (11)	2,193	88.3%	0	+401 (+2.68)	+50 (+0.33)
Hunt R1	25 (25)	531	74.0%	0	+180 (+1.21)	+180 (+1.21)
Hunt2 R1	32 (32)	677	85.4%	0	+143 (+0.96)	+136 (+0.91)

Table 4-6: AI rounds on baseline v2 — events and percentage points of the 14,935 TCP/UDP attack events; rules counted over both rounds

Capture	Reasoning	Turns	Duration	Input tokens	test_rule	Rules saved /

	carried				calls	report
3	No	71	29m 5s	5,873,072	3	7 / yes
1 (first attempt)	No	100	12m 13s	8,423,639	7	0 / no — turn limit
1	Yes	90	10m 40s	5,198,227	7	9 / yes
10	Yes	93	12m 18s	8,593,548	8	9 / yes
3 (hunt2)	Yes	68	10m 30s	6,050,275	8	9 / yes
1 (hunt2)	Yes	143	19m 22s	14,963,616	13	16 / yes
10 (hunt2)	Yes	54	7m 20s	3,120,460	3	7 / yes

Table 4-7: Hunt and hunt2 runs (DeepSeek V4 Pro, web interface)

The hunt round was run through the web interface with the new profile selector (Table 4-7). On capture 3 the model mapped coverage, concentrated on HTTP as the protocol with the most external traffic and attacker-controlled fields, and found rare values that pointed to specific exploits the baseline did not detect: the D-Link router backdoor user agent (CVE-2013-6026), three HP OpenView Network Node Manager CGI attacks, a Cisco Secure ACS overflow, a vulnerable password-change CGI and a PHP webshell drop. Its seven rules all passed the gate, one after a correction.

The first attempt on capture 1 failed in a revealing way. Its rules passed test_rule, but in the remaining turns the model re-tested rules that had already passed, re-checked coverage it had already established, never saved anything and reached the 100-turn limit; the transcript contained tool calls but no reasoning at all. Checking each layer in turn — engine, server, browser, then the raw API — located the cause: DeepSeek V4 Pro was answering in its thinking mode, returning its reasoning in a separate reasoning_content field with an empty content field, and the provider adapter read only content. The model's reasoning was therefore neither logged nor sent back with the next turn, so it began every turn without its own plan. The adapter now streams the reasoning to the interface and the transcript and returns it with each turn; the hunt profile's turn budget was also raised to 150 (the fixed runs used 90 and 93 turns, so the limit did not bind). With the fix, capture 1 finished on its own in 90 turns and capture 10 in 93, each with a full report and the model's reasoning recorded on every turn.

The rules written after the fix are of a different kind: instead of single exploit attempts they target techniques — path traversal in its raw, overlong-UTF-8 and encoded-backslash forms, cross-site scripting and PHP code in URIs and user agents, format strings in HTTP, FTP and SMTP — and two behaviours: an over-long SIP INVITE URI and SMB file names padded with fuzzing characters. The model again dropped candidates it judged unreliable: randomised POP3 and SIP logins and SMTP spam as

indistinguishable from legitimate external users, and an LDAP enumeration whose only distinguishing string was specific to this laboratory. Table 4-8 lists all 25 rules.

The hunt2 runs, all with the reasoning fix, took 68, 143 and 54 turns and saved 32 rules. The model used the keyword tool on every capture — 24 lookups in all — and none of its rules failed to load, against two load failures in the hunt condition's capture 10. Its rules use the protocol buffers the tool listed (ftp.command_data, ldap.request.dn, dns.query.name and a parser event, app-layer-event:smb.duplicate_negotiate) rather than raw bytes. Two of the 32 are not genuine: an LDAP modify-DN request from outside HOME_NET (topology) and an LDAP bind keyed on this laboratory's domain string (dataset indicator) — the first non-genuine rules the hunt procedure produced.

SID	Rule (model's message)	Goal (model's ADS comment)
7800001	EXPLOIT D-Link DIR-100 auth bypass xmlset_roodkcableoj28840ybtide User-Agent	Detect D-Link/Planex router auth bypass exploit using xmlset_roodkcableoj28840ybtide User-Agent
7800002	EXPLOIT HP OpenView NNM jovgraph.exe displayWidth overflow attempt	Detect HP OpenView NNM jovgraph.exe displayWidth buffer overflow exploit
7800003	EXPLOIT HP OpenView NNM nnmRptConfig.exe format string attempt	Detect HP OpenView NNM nnmRptConfig.exe format string exploit
7800004	EXPLOIT Cisco Secure ACS CSUserCGI.exe Logout overflow attempt	Detect Cisco Secure ACS CSUserCGI.exe Logout buffer overflow
7800005	SERVER-WEBAPP chetcpasswd.cgi access	Detect access to the vulnerable chetcpasswd.cgi password changer
7800006	EXPLOIT HP OpenView NNM OvWebHelp.exe Topic overflow attempt	Detect HP OpenView NNM OvWebHelp.exe Topic buffer overflow
7800007	SERVER-WEBAPP PHP webshell creation via URL parameter	Detect PHP webshell creation through a URL parameter carrying PHP passthru code
7800008	SERVER-WEBAPP Unicode overlong directory traversal attempt	Detect IIS-style overlong UTF-8 directory traversal (%c0%ae%c0%ae encodes a dot)
7800009	SERVER-WEBAPP encoded backslash traversal attempt	Detect URL-encoded backslash directory traversal (..%5c) in HTTP requests
7800010	SERVER-WEBAPP HTTP URI format string attempt	Detect HTTP URI format-string probes using huge width specifiers (%99999999d/%99999999s)
7800011	SERVER-WEBAPP phpinfo in HTTP User-Agent	Detect phpinfo injection carried in the HTTP User-Agent header
7800012	SERVER-WEBAPP XSS in HTTP User-Agent	Detect reflected/script injection probes carried in the HTTP User-Agent header
7800013	SERVER-WEBAPP encoded XSS in HTTP URI	Detect URL-encoded triple less-than (<<<) used by encoded XSS/HTML injection probes
7800014	FTP RETR path traversal attempt	Detect FTP RETR commands using path traversal (/../) to read files outside the FTP root
7800015	SERVER-WEBAPP encoded backslash traversal attempt raw dots	Detect raw-dot encoded-backslash directory traversal (..%5c) in HTTP requests
7800016	FTP command format string attempt	Detect FTP commands carrying huge-width

		printf format specifiers (%99999999d) as login/command arguments
7800017	SERVER-WEBAPP HP HPEinc remote file inclusion attempt	Detect HP HPEinc remote file inclusion
7800018	EXPLOIT Novell ZENworks UploadServlet WAR upload attempt	Detect Novell ZENworks UploadServlet WAR upload RCE
7800019	SERVER-WEBAPP HTTP path traversal attempt	Detect raw HTTP path traversal
7800020	SERVER-WEBAPP XSS in HTTP URI	Detect raw XSS in HTTP URI
7800021	DNS BIND authors query attempt	Detect BIND AUTHORS.BIND reconnaissance query
7800022	SIP INVITE with excessively long URI	Detect SIP INVITE with excessively long URI
7800023	TLS certificate subject null byte	Detect TLS certificate subject containing a null byte
7800024	SMTP RCPT TO format string attempt	Detect SMTP RCPT TO format string
7800026	SMB NT_CREATE_ANDX fuzz filename	Detect SMB NT_CREATE filename fuzzing with repeated punctuation

Table 4-8: Rules written in the hunt condition (captures 3, 1 and 10); every rule passed test_rule before it was saved

V. Analysis and Discussion

The analysis of this section looks at various aspects of the triage execution, such as model comparison, the quality of the AI-written rules compared to the stock community rules, the impact of raw data versus suppressed data, and cross-day generalization behaviour. It then reports the ground-truth evaluation of every AI-authored rule on both datasets, the rule audit, the context experiment, and threats to validity.

Introduction

The results of the analyst workflow performed by the AI triage engine in Section IV indicate a successful execution of the analyst workflow in all four attack days. This section will look at the results and what they tell us about the research question: Can a generative language model replace the human analyst in the alert-handling loop, and when?

Model Comparison

For the credential brute-force capture (Tuesday) , the only capture both model families were run against .Claude Opus 5 and DeepSeek V3 reached the same substantive conclusions across all attack categories. No attack was detected by one model and not by the other. This agreement is weaker evidence than it appears. The only DeepSeek V3 run that completed the Tuesday triage (triage-2026-09-04-03-07-23.txt) read the Tuesday ground truth from the project's notes before concluding; the two earlier V3 runs exhausted their 30-turn limit before the count_by tool

existed. Claude's run was a CLI session that was not logged, so the same check cannot be made. Neither result can be shown to be independent of the labels. The Wednesday tuned pass was made with DeepSeek V3, whereas the Wednesday raw pass, Thursday, and Friday were made with DeepSeek V4 Pro. There were variations among the models in their level of efficiency, expense, and rules complexity.

Metric	Claude Opus 5	DeepSeek V3	DeepSeek V4 Pro
Captures triaged	Tuesday	Tuesday, Wednesday (tuned)	Wednesday (raw), Thursday, Friday, UNSW-NB15 (6 captures)
Turns (Tuesday tuned)	Not logged (CLI run)	44 (label-informed run; see Threats to Validity)	—
Turns (Wednesday)	—	38 (tuned)	81 (raw, incl. V3 rule validation)
Turns (Thursday)	—	—	62 (tuned), 47 (raw)
Turns (Friday)	—	—	58 (tuned), 63 (raw) (tuned input duplicated; see Section IV)
Key findings (Tuesday)	FTP + SSH brute force	FTP + SSH brute force (identical) — completed run read ground truth from project notes (see Threats to Validity)	—
Rules written	2 (SID 9500001–9500002)	8 (SID 9000001–9000005, 3000001–3000003; 3000001–3000002 later	14 new (SID 3000004, 7600001–7600007, 7600101–7600103,

		deleted — §V)	9500003, 9600001–9600002); also rewrote V3's SID 3000003; later 6 ARES rules (CIC-IDS2017) and 18 UNSW-NB15 rules
Rule approach	Campaign-level thresholds	Stateful chains, tool fingerprints	Forensic analysis, parser workarounds
Turns (UNSW-NB15)	—	—	43–75 per capture (6 runs)

Table 5-1: Model comparison across triage runs. Turn counts are from session transcripts.

This was where DeepSeek V4 Pro really excelled, showing a remarkable ability to diagnose and overcome a Suricata parser limitation that Claude and DeepSeek V3 didn't have. When the FAKEVERB detection rule did not detect anything, V4 Pro detected that the HTTP parser in Suricata doesn't expose non-standard methods to the HTTP. method buffer, then rewrote the rule to use raw TCP payload matching, getting 463 hits while the previous approach got zero hits.

Model-agnosticism is a property of the design — one prompt, one tool set and one loop behind a provider adapter — rather than a result this evaluation establishes. The one head-to-head comparison is compromised (above), and every other capture was triaged by a single model. Wednesday offers only indirect evidence: V3 on the tuned pass and V4 Pro on the raw pass identified the same attacks from different alert sets. A controlled comparison — two models, one capture, identical locked-down environments — remains future work. A formal cost comparison should include each provider's published pricing and should include input/output token ratios, Token usage is recorded per turn in every transcript (for example, 2.7–6.5 million input tokens per UNSW-NB15 run), but each provider's pricing at the time of the runs was not captured, so a monetary comparison is left as future work.

AI-Authored Rules vs Stock Community Rules

AI-authored rules outperformed stock ET Open rules in several specific areas, as summarised in Table below:

Attack	Stock ET Alerts	AI Rule Alerts	Improvement
Slowloris/ Slowhttptest	3 (SID 2014103)	187 (SID 3000001, later removed — see §V Model Errors)	62×
FAKEVERB DoS method	0 (no rule)	463 (SID 3000003)	New detection
Hulk GET flood	0 (no rule)	95 (SID 3000004)	New detection
HTTP login brute force	0 (no rule)	8 alerts, all on labelled brute-force flows (SID 7600001)	New coverage
Reverse shell (.25)	0 (missed)	4 alerts: 2 on Infiltration flows, 2 on benign (SID 7600003)	New coverage
SMB lateral sweep	Parser noise only	Rule fails to load (SID 7600002)	None
Friday HTTP DDoS	40 (SID 2019348)	11 campaign confirmations (SID 9600001)	Campaign-level
Friday SYN sweep	Known ports only	13 confirmations (SID 9600002)	Broader coverage
SSH brute force (Tuesday)	60 SSH-Patator flows matched (1.0%)	574 alerts, all on SSH-Patator flows (SID 9000004)	1.0% → 28.4% of flows
ARES botnet C2 (Friday)	0	15 alerts, all on Bot flows (SID 9700001–9700003)	New detection
FTP “data staging” (Thursday)	—	768 alerts, none on attack flows (SID 7600004)	False-positive rule

Table 5-2 -Detection coverage: AI rules vs stock ET Open

The improvements can be divided into three categories. Firstly, that quantitative improvement: the AI rule caught 187 attacks of the same type (Slowloris) while the human rule caught only 3, a 62× increase. Second, new coverage where the stock ruleset had none: FAKEVERB, the Hulk flood, the HTTP login brute force, the reverse shell and the ARES botnet, each now confirmed by ground-truth replay (Table 5-2); the SMB lateral-sweep rule does not load and provides no coverage. Third, campaign-level detection is only context-aware, as a stock rule-based per-packet approach cannot (stateful brute-force chains, sustained flood confirmation).

Ground-Truth Evaluation (Experiment 2)

Method. Every alert was joined to the datasets' labels. For CIC-IDS2017 an alert matches a labelled flow if it shares the flow's 5-tuple (in either direction) and its timestamp falls within the flow's lifetime plus 120 seconds either side; the slack is needed because the label timestamps have one-minute resolution, use a 12-hour clock without AM/PM, and are in Atlantic Daylight Time. For UNSW-NB15 an alert matches a ground-truth event if it shares its IPs, ports and protocol within the event's time window (±2 seconds). UNSW results are headlined for TCP/UDP events only: other protocols carry no ports, IP-only matching is sensitive to the tolerance, and TCP/UDP results were identical at ±2 and ±10 seconds. Suricata's own engine and decoder events (SIDs 2200000–2299999) are excluded from the stock baseline, because in raw CIC captures the invalid-checksum event fires on nearly every packet and would count every flow as detected. Tuesday and Thursday were replayed with every CIC rule loaded; the Wednesday and Friday captures had been removed from the working environment to free storage for UNSW-NB15, so their AI rules are scored from the validation logs recorded during triage. The scripts are in src/eval/ of the repository.

Detection on CIC-IDS2017. Figure 5-1 shows, for each labelled attack, the share of flows matched by at least one alert. The AI rules add detection where the stock ruleset had little or none: SSH-Patator rises from 1.0% to 28.4%, DoS Slowhttptest from 0% to 9.1%, and web brute force, Bot and DoS Hulk from 0% to between 0.4% and 1.0%. The low absolute values for volumetric attacks are a property of the metric: the AI rules are deliberately campaign-level and fire once per threshold window, while the labels count every flow — 231,073 of them for DoS Hulk. Category coverage and precision describe these rules better. DoS GoldenEye and DoS slowloris are detected by neither rule set; the model's 'Slowloris' rule in fact matches flows labelled DoS Slowhttptest, a confusion of two tools that its own rule text reveals.

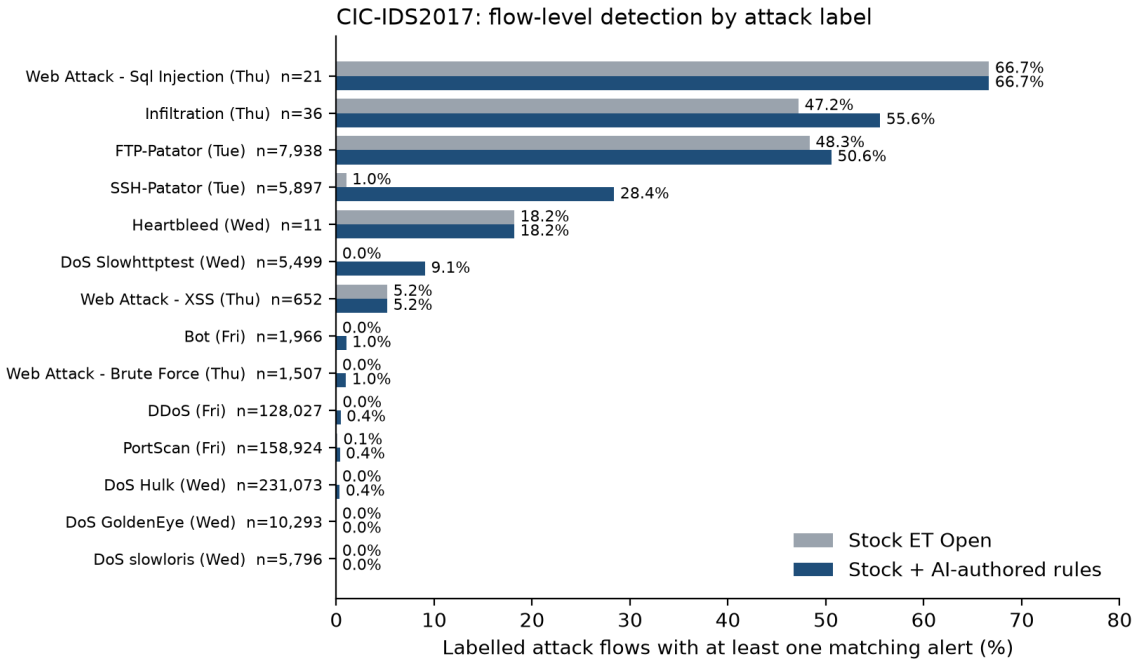


Fig. 5-1 : CIC-IDS2017 — labelled attack flows matched by stock ET Open and by stock plus AI-authored rules

Detection on UNSW-NB15. Across the six captures the AI rules raise event-level detection of TCP/UDP attacks from 9.2% to 15.5%, a 68% relative increase (Figure 5-2). Reconnaissance rises from 18.7% to 36.2% and Exploits from 9.9% to 19.1%. Alerts on traffic with no attacker involved did not increase (749 stock, 746 with AI rules): every alert of every AI rule involves the attacker range. The absolute values are low even for the stock rules because the generator launches thousands of one-off exploit attempts, many with no signature in any ruleset. Re-scored by attribution within a single replay (Section V, Attribution and the Topology Trap), the AI rules account for +6.21 points of events no other rule caught, +5.90 of them from signature and behavioural rules: against stock ET Open the gain is genuine, unlike the later rounds against the strengthened baseline.

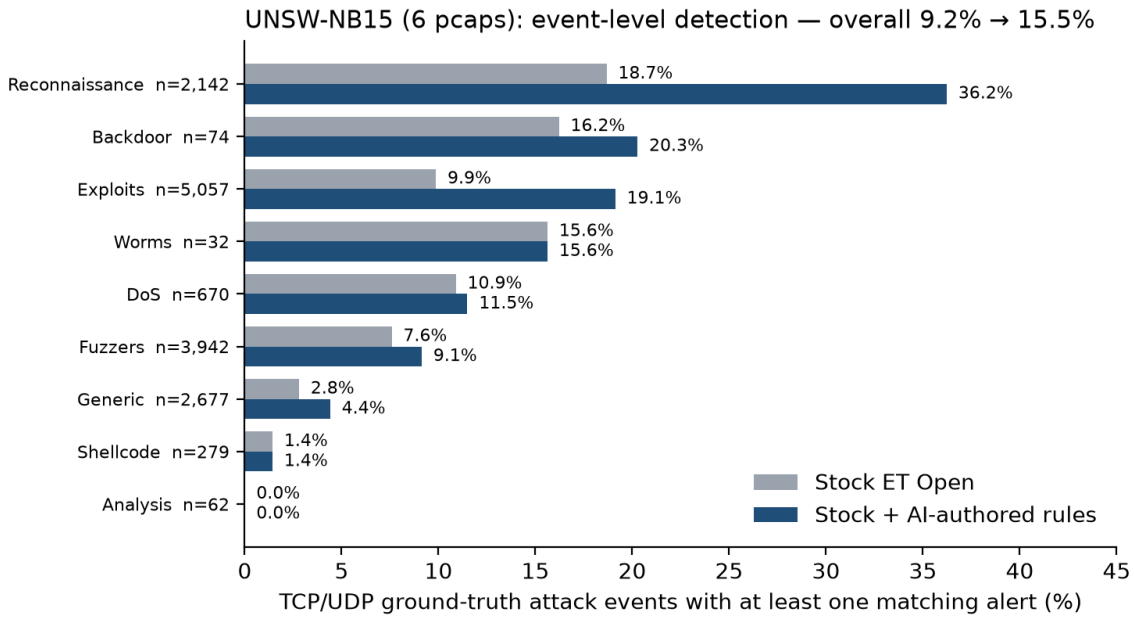


Fig. 5-2 : UNSW-NB15 — TCP/UDP ground-truth attack events matched by stock ET Open and by stock plus AI-authored rules

Precision. Of the stock rules' label-matched alerts on the two noisiest captures (Wednesday and Friday raw), only 6.1% fall on attack flows; for the AI rules the figure is 99.4% (Figure 5-3). Over all four days the stock rules reach 81.2%, because Tuesday and Thursday stock alerts are mostly on real attacks, and the AI rules 78.0%. Almost all of the AI rules' false positives come from a single rule, the FTP 'data staging' rule SID 7600004; excluding it and the disputed internal-sweep rule SID 7600007 gives 99.6%.

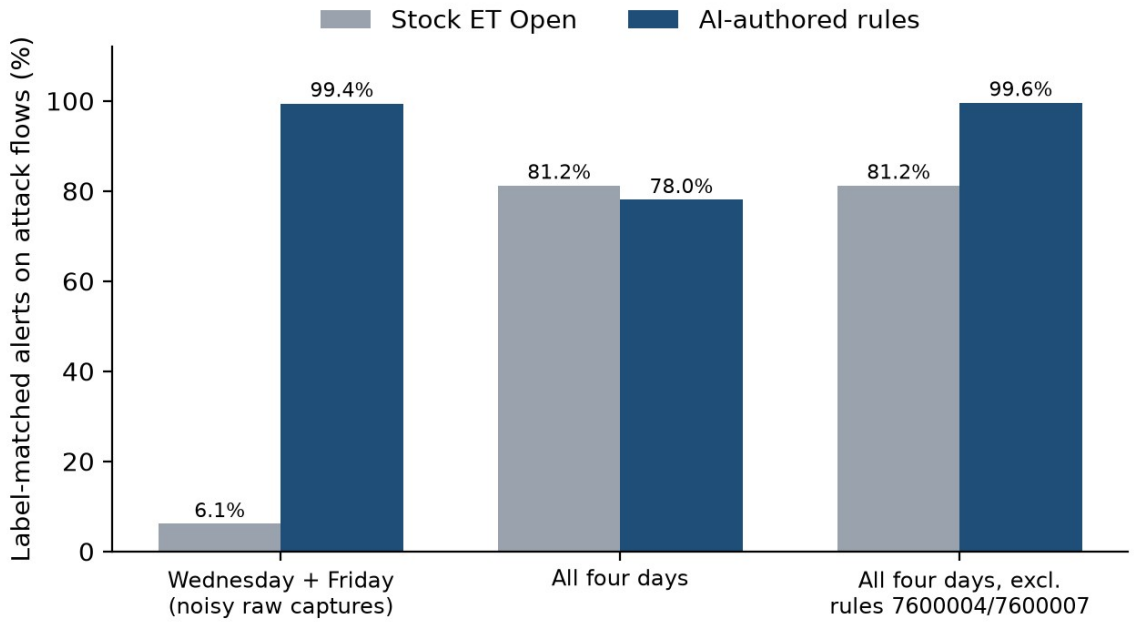


Fig. 5-3 : Share of label-matched alerts that fall on attack flows — stock ET Open versus AI-authored rules

Category coverage. Figure 5-4 summarises coverage as the number of labelled attack categories with at least one true-positive alert, and the number the model named in its findings. On CIC-IDS2017 the stock rules cover 8 of 14 categories, the stock and AI rules together 12, and the model's findings 13 (counting DoS slowloris, which the model reported together with DoS Slowhttptest). On UNSW-NB15 all three reach 8 of 9, with Analysis missed by all; there the AI rules add depth within categories rather than new categories.

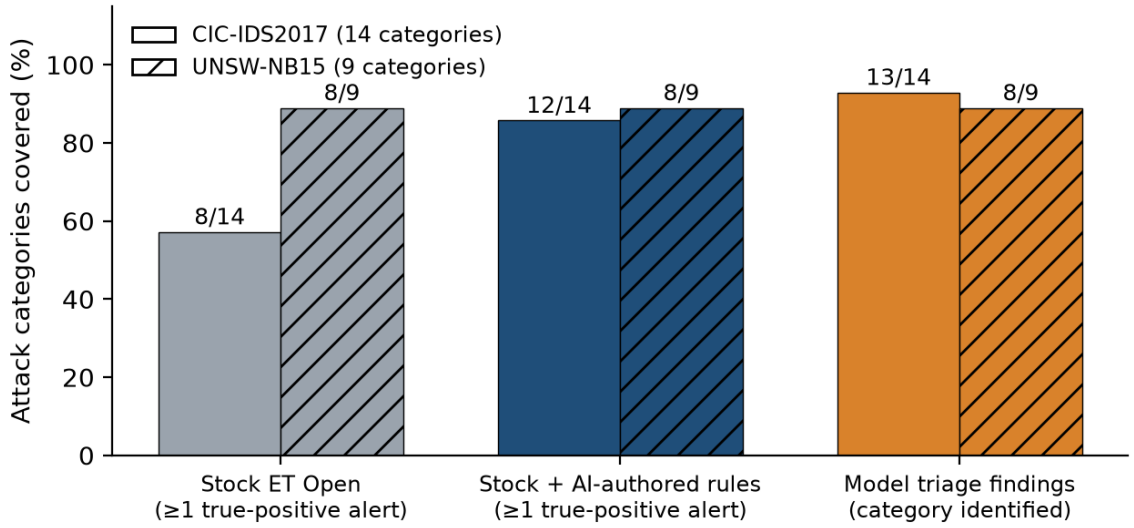


Fig. 5-4 : Attack categories covered by stock rules, by stock plus AI-authored rules, and by the model's triage findings

Comparison with human-authored rules. ET Open is written and maintained by human experts, so the comparison above is a comparison of AI-authored rules with human-authored ones on the same traffic. On time, Scarfone and Mell (2007) describe manual rule development as taking hours to days; each triage run in this project covered an entire capture and produced validated rules in 9 to 111 minutes. A direct comparison with a human analyst performing the same triage was not carried out and is proposed as future work.

Rule Audit

Every rule the models authored was traced to its session and checked for whether it loads, whether it was ever replayed, and whether it fired (Table 5-3, Figure 5-5; the full list is in Appendix D). The contrast between the datasets is the main result. All 18 UNSW-NB15 rules load and fire. Of the 32 CIC-IDS2017 rule definitions, 16 fired on replay; three fail to load, one loads but can never fire, the model deleted two, four Tuesday rules never fired when replayed, and six were never replayed at all. The audit covers the CIC-IDS2017 and first UNSW-NB15 rules; the 53 rules written in the Experiment 3 rounds are classified in Attribution and the Topology Trap.

The difference is environmental. During the CIC-IDS2017 triage the Suricata configuration listed only three of the ten generated rule files, and the model is not permitted to edit it. New rule files were therefore never loaded by the model's own validation runs, which reported success while testing nothing. The model worked around this in two sessions by copying its rules, under new SIDs, into a file that was loaded — the origin of SIDs 9600003–9600004 and of the ARES rules inside Claude's Tuesday rule file. For UNSW-NB15 the configuration listed the model's rule file from the outset, and every rule was genuinely validated. An agent's self-validation is only as good as the environment it validates in.

Rule status	CIC-IDS2017	UNSW-NB15
Fired on replay	16	18
Replayed, never fired	4	0
Never replayed (file not loaded)	6	0
Loads, can never fire	1	0
Fails to load	3	0
Deleted by the model	2	0
Total rule definitions	32	18
of which copies or duplicates	2	4
of which superseded drafts	3	0

Table 5-3: Audit of AI-authored rule definitions

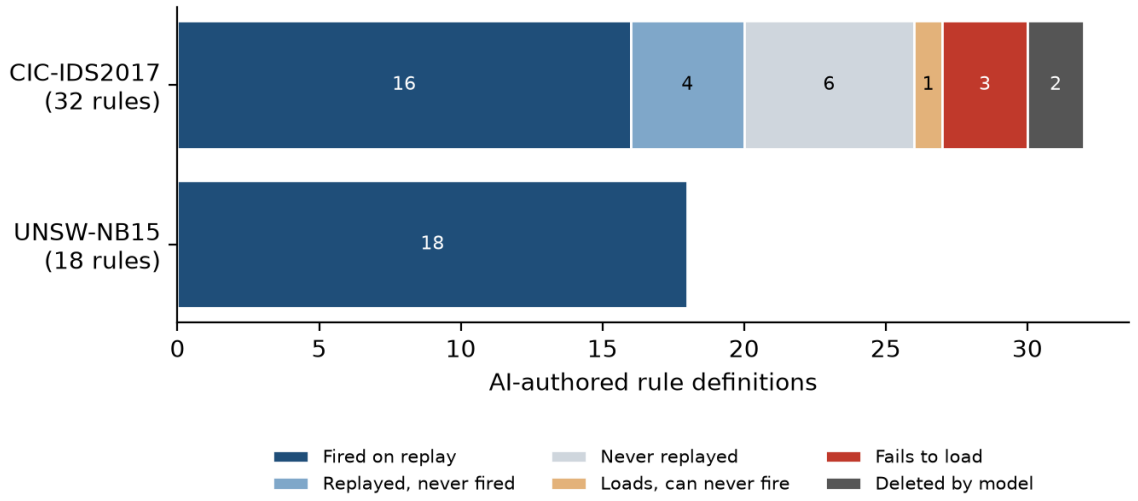


Fig. 5-5 : Status of every AI-authored rule definition

Raw vs Suppressed Data

All days showed a consistent result: detections in the raw (unsuppressed) pass were not observed in the tuned pass. The second reverse shell host, Thursday (192.168.10.25), did not generate a stock alert and was discovered only via flow-level analysis, as the smaller dataset from the

tuned pass did not trigger the model to take action. Likewise, the Wednesday 'Hulk' flood was found from the raw pass, as the tuned pass had removed the low-level noise.

This is a practical compromise. The volume of alerts processed by the model is greatly reduced, which is why cost overheads are also significantly lowered, compared to the raw pass (which processes 2,310,105 alerts, while the tuned pass processes only 381). However, the raw pass gives the model full visibility and enables identification of coverage gaps that cannot be detected when the noise floor is filtered beforehand. In production deployments, this translates to a two-pass approach: a tuned pass for cost-efficient, regular production maintenance, and a raw pass now and then to uncover any coverage gaps. On UNSW-NB15 the distinction largely disappears: raw captures produced 625–997 alerts each, because the invalid-checksum alerts that dominate raw CIC-IDS2017 volume are an artefact of that capture, not of raw mode itself.

Cross-Day Generalization

SID 9500002, written by Claude for Tuesday's SSH brute force, fired on Friday traffic from the same attacker; all 79 of its label-matched Friday alerts fall on PortScan flows, so it generalised to the SSH phase of the port scan rather than to a second brute-force campaign. Stronger evidence comes from UNSW-NB15: rules written on one capture fired on the others (13 of 15 on capture 14). Two CIC Heartbleed rules also matched 495 UNSW attack events, but they are topology-style rules (Section V), so that match is not evidence of transfer.

The interim report also described a cross-capture correlation: Tuesday's cracked FTP credentials (iscxtap/1234) reused for Thursday data staging. The ground-truth evaluation does not support it (Section IV): identical uploads come from every workstation on both days and are labelled benign. The model constructed a plausible narrative from routine activity.

Model Errors and Rule Provenance

An instructive error in model reliability is found in the triage logs. The three rules created by DeepSeek V3 for the Wednesday Slowloris campaign (SID 3000001-3000003) were written for the tuned pass. When DeepSeek V4 Pro ran the raw pass, which loaded in the rules from V3 as well as the stock ruleset, it deleted both of these rules from the file and output in its logs that they were stock ET Open rules that should not be re-declared. However, this is not true, and the output for grep -c 'sid:3000001' rules/suricata.rules and grep -c 'sid:3000002' rules/suricata.rules is both returned as zero matches. The SID range

3000001–3000002 is not included in the ET Open ruleset; both rules were written by V3 and deleted by V4 Pro (hallucinated provenance claim).

In addition, V4 Pro rewrote SID 3000003 in place (changing from http. method to raw TCP depth:8 matching), in contrast to the project convention that prescribes that changes to the rules are made with either care or a tool, but are never made by hand (and changes to rules are a finding, not a correction to apply silently). The original rule text for all three SIDs is in the triage transcripts (authoring: triage-2026-09-04-04-10-26.txt; deletion: triage-2026-09-15-11-26-28.txt).

The importance of this episode lies in two aspects. For one thing, it shows that models can make bold statements about the state of an external system (the contents of a 44MB ruleset) that are hard to verify in real time. Second, the rules that were removed were some of the most effective detections that were generated – the SID 3000001 was triggered 187 times on Slowloris traffic, while the closest stock rule was only triggered 3 times. The complete rule inventory is given in the Rule Audit and Appendix D.

The ground-truth evaluation and later runs exposed further errors of the same general kind — confident output that was not checked against reality:

Rules that do not work. Three Thursday rules use keywords Suricata 8.0.6 rejects, and one Heartbleed rule depends on an event that does not exist. The model reported all four as written and, in the Thursday case, as validated — its validation run had replayed a small test capture.

Misread traffic. The FTP 'data staging' finding turned routine uploads into exfiltration, and the 'Slowloris' rule matches Slowhttptest traffic.

Skipped procedure. One UNSW run wrote rules without validating them, and one read its rule file and still re-wrote two existing rules.

Inconsistent verdicts. The same model reached opposite verdicts on the same evidence in different runs: five UNSW runs judged the Heartbleed rules correct and one proposed suppressing them, and two runs split on whether a shellcode signature was a false positive.

Duplicated input. The Friday tuned triage analysed an eve.json file containing two appended Suricata runs, so any count it quoted from that run is doubled.

Attribution and the Topology Trap

Crediting the right rules. The first scoring of the AI rounds compared a fresh replay with the AI rules against the stored baseline replay. Because Suricata's threshold and flow logic depends on timing, two replays of the same capture with the same rules do not produce identical alerts, and the difference was counted as gain: for the hunt round this method reported +323 events, of which only 180 were matched by a hunt rule. All rounds were therefore re-scored by attribution — an event counts only if an AI-rule alert matches it and no other alert in the same replay does (Figure 5-6).

Topology rules. Every AI rule was then classified by its text as signature (attack content), behavioural (a rate or a protocol misuse), dataset-indicator (a string that identifies this testbed), or topology: ordinary protocol activity from outside HOME_NET with no attack constraint, such as an SMTP MAIL FROM, a POP3 login, a SIP REGISTER or an FTP login. In UNSW-NB15 every external host is an attacker, so a topology rule scores perfectly here and would alert on every legitimate external user of a real network. Figure 5-7 shows the composition by condition.

Most of the apparent gain was topology. Plain round 2 added +11.31 points of attributable detection, but only +0.08 from signature and behavioural rules; 10 of its 16 rules are topology rules. The research condition wrote 5 topology rules of 12 and reached the largest genuine gain, +0.33 points of the two triage-driven conditions. Their genuine gains differ by a few dozen events, too small to rank them. The hunt condition wrote 0 topology rules and added +1.21 points, and the hunt2 condition +0.91 — the only gains clearly above the noise (following subsections).

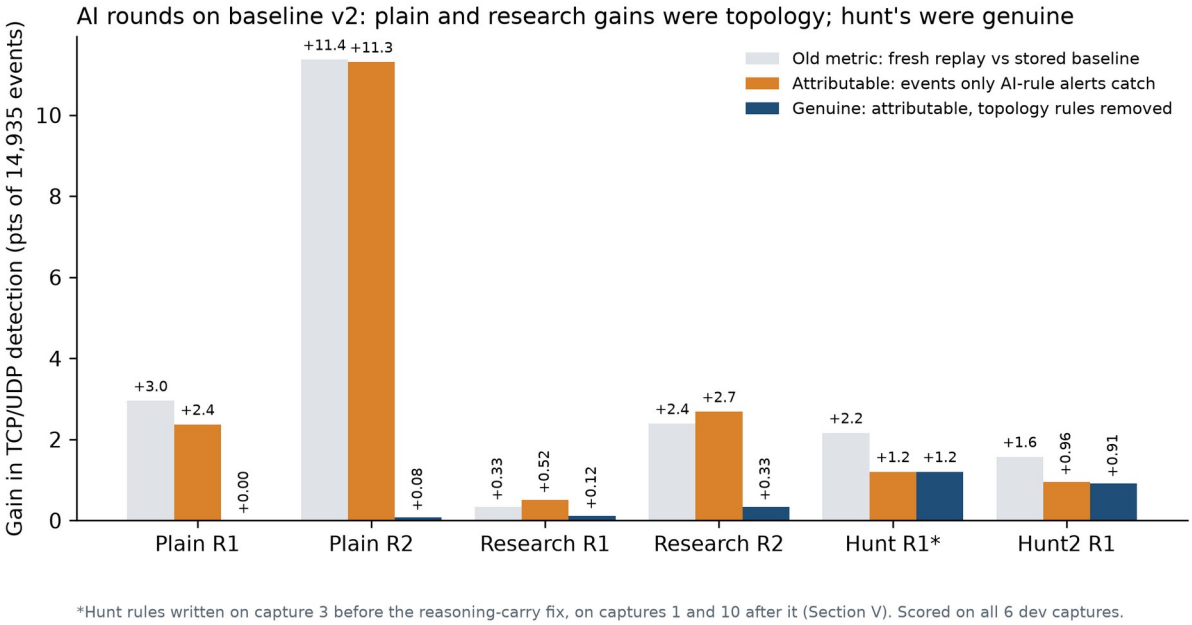


Fig. 5-6 : AI-round gains under the old metric, by attribution, and with topology rules removed

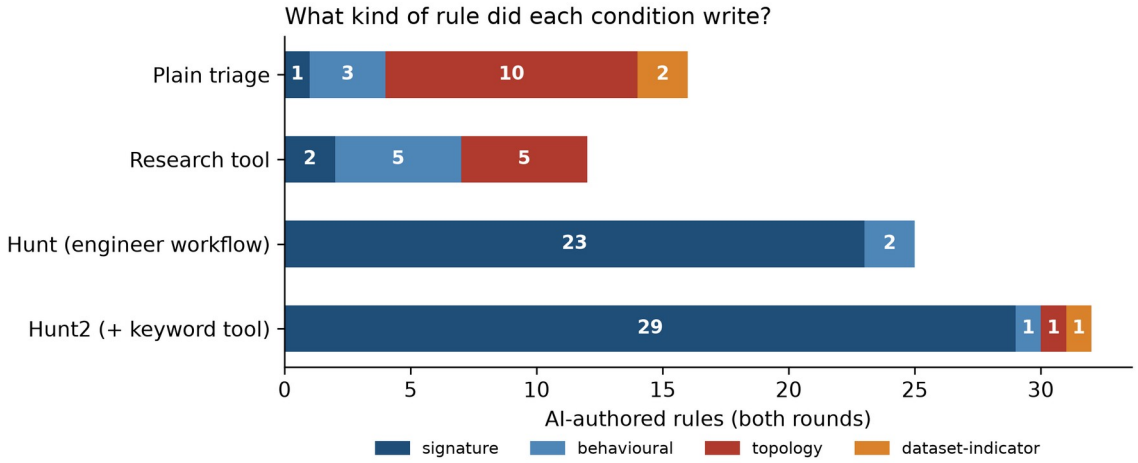


Fig. 5-7 : Classes of AI-authored rules by condition

The Detection-Engineering Workflow

The hunt condition changed the kind of rule the agent wrote. All 25 of its rules are content signatures for named vulnerabilities; none is a topology rule; 74.0% of their alerts match a labelled attack and 0 fall on hosts with no attacker involved. The gate behaved as designed: on capture 3 the topology example fails because, widened, it matches 1,668 times between internal hosts, while the traversal example raises 10 alerts either way. The gate has a structural blind spot: it can only test protocols that internal hosts also use, and in UNSW-NB15 no internal host uses POP3, SIP or SMB (Figure 5-8).

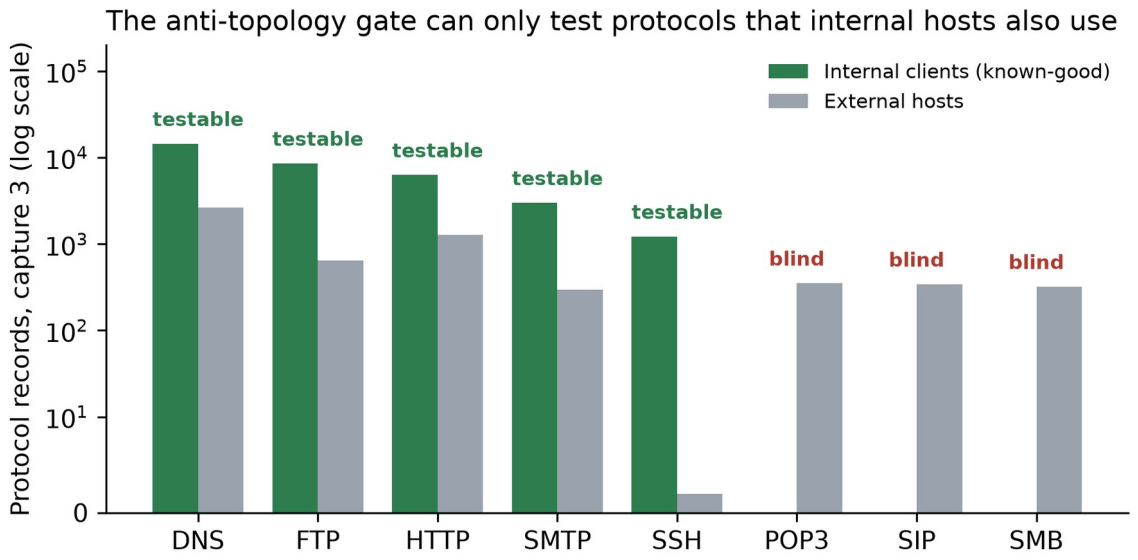


Fig. 5-8 : Protocol records by source role on capture 3 — the gate can only test protocols used by internal hosts

Whether the rules transfer depends on when they were written (Figure 5-9). The seven rules from capture 3, written before the reasoning-carry fix, each fired exactly once, on capture 3: several key on the exact filler of one exploit attempt — 'Logout+aaaa', 'displayWidth+1AAAA' — rather than on the property that makes it an overflow. In Atkinson's (2020) terms they detect the tool's surface, not the capability, and UNSW-NB15's generator launches each exploit with its own payload. The 18 rules written after the fix target techniques and behaviours, and most fire on captures they were never written on. Together the 25 rules added +180 attack events (+1.21 points); on development captures 4, 5 and 14, on which no condition wrote rules, the hunt rules added +1.38 points against +0.09 for plain and +0.22 for research. The largest single contributor is the behavioural SMB fuzz-filename rule; SMB is one of the protocols the gate cannot test, so its only benign check is the hold-out's normal-traffic captures, on which it raised no alert.

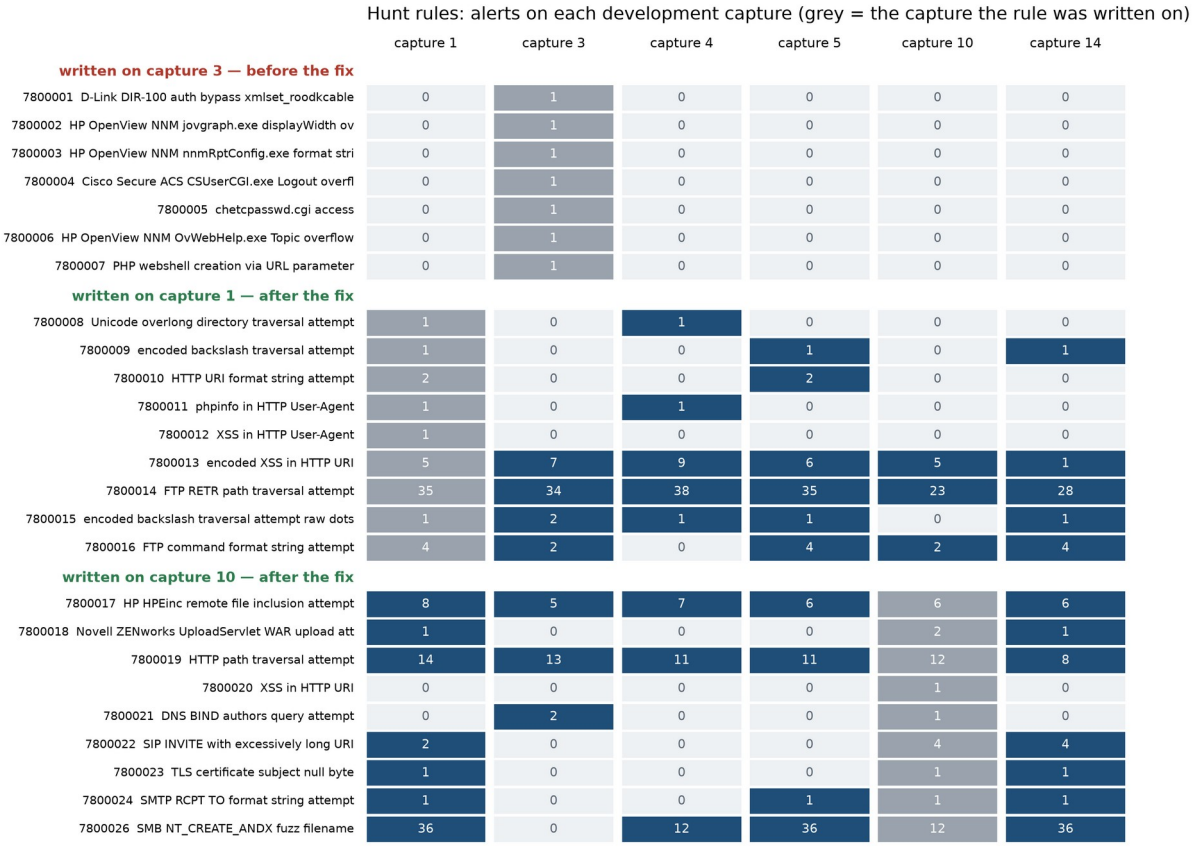


Fig. 5-9 : Alerts of each hunt rule on each development capture, grouped by the capture it was written on

Hold-out Evaluation

Every result above comes from the six development captures, on which rule families were selected and AI rules were written. Final claims need traffic that neither the team nor the model had looked at, so five captures from the other UNSW-NB15 day, 22 January 2015, were held out. That day's ground truth covers only 11:00–14:00 UTC; captures 1, 3 and 6 fall inside the window and measure detection, while captures 25 and 50 contain no labelled attacks and serve as unseen normal traffic, on which every alert is a false-positive candidate. Captures 1, 25 and 50 were chosen for spread across the day before any label was examined; 3 and 6 were added, by time only, when it became clear that 25 and 50 fall outside the attack window. The rule for selecting what is tested — baseline v2 plus non-topology AI rules — was fixed before any hold-out replay, and every AI rule file was frozen (copied to rules/snapshots/) before the AI rules were replayed on the hold-out, once.

The frozen baseline v2 detects 29.5% of the 4,634 hold-out TCP/UDP attack events, and 65.3% of its alerts match an attack, against 24.9% and 57.8% on the development captures (Figure 5-14). The family selection therefore generalises to unseen traffic rather than reflecting tuning to the development day.

The frozen AI rules were then replayed on the hold-out (Table 5-4, Figure 5-10). The pattern of the development set held. Plain round 2's large attributable gain was again almost entirely topology: genuine +0.11 points. Research added +0.28. The hunt rules added +71 events, +1.53 points — slightly more than on the development set — with 91.6% of their alerts on labelled attacks. No AI rule of any condition raised an alert on a host with no attacker involved, or on either benign-only capture. Hunt2 added +1.06 points. Taking all four conditions together, the genuine events they add come to approximately +101 (+2.2 points), and the two hunt conditions together +87 (+1.9); because each condition was attributed within its own replay, these unions are approximate.

Condition	Rules fired	AI-rule alerts	Matching an attack	Attributable gain	Genuine gain	Alerts on benign captures
Plain R2	14 of 16	966	98.3%	+532 (+11.48)	+5 (+0.11)	0
Research R2	10 of 12	835	84.0%	+155 (+3.34)	+13 (+0.28)	0
Hunt R1	17 of 25	155	91.6%	+71 (+1.53)	+71 (+1.53)	0
Hunt2 R1	22 of 32	177	100.0%	+51 (+1.10)	+49 (+1.06)	0

Table 5-4: Frozen AI rules on the hold-out (captures 1, 3, 6: 4,634 TCP/UDP events; benign-only captures 25, 50)

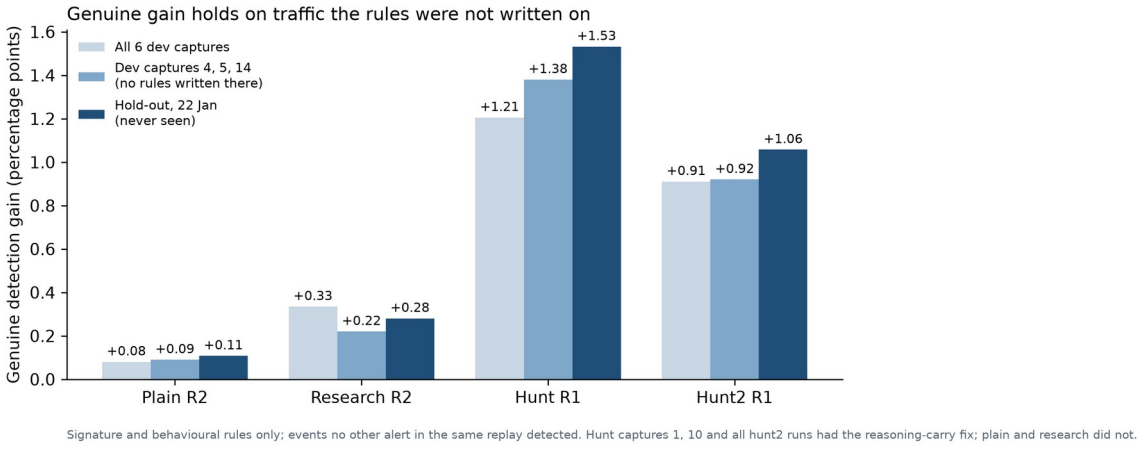


Fig. 5-10 : Genuine gain on all development captures, on development captures without rules, and on the hold-out

Adding a Keyword Reference (Hunt2)

Hunt2 added +136 genuine events on the development captures (+0.91 points), +0.92 points on captures 4, 5 and 14, and +1.06 points on the hold-out, with 100.0% of its hold-out alerts on labelled attacks and none on benign traffic. That is consistent and generalises, but it is below the hunt condition and below the expectation recorded before the run (+1.2 to +2.0 points). The keyword tool did what it was built to do — no rule failed to load — yet it did not raise detection.

Figure 5-11 shows why. The two conditions found largely different attack events: of the hunt's 180 and hunt2's 136 genuine development events, 82 are common, 98 were found only by the hunt (its SMB fuzz-filename rule alone contributes 59) and 54 only by hunt2 (its WebDAV PROPFIND rule for Windows DLLs, 36). Which attack families a run chooses to hunt decides its result, and that choice varies more between runs of the same procedure than the keyword tool changes it. With one run per capture per condition, the difference between hunt and hunt2 is within run-to-run variation; the union of the two conditions' rules is larger than either.

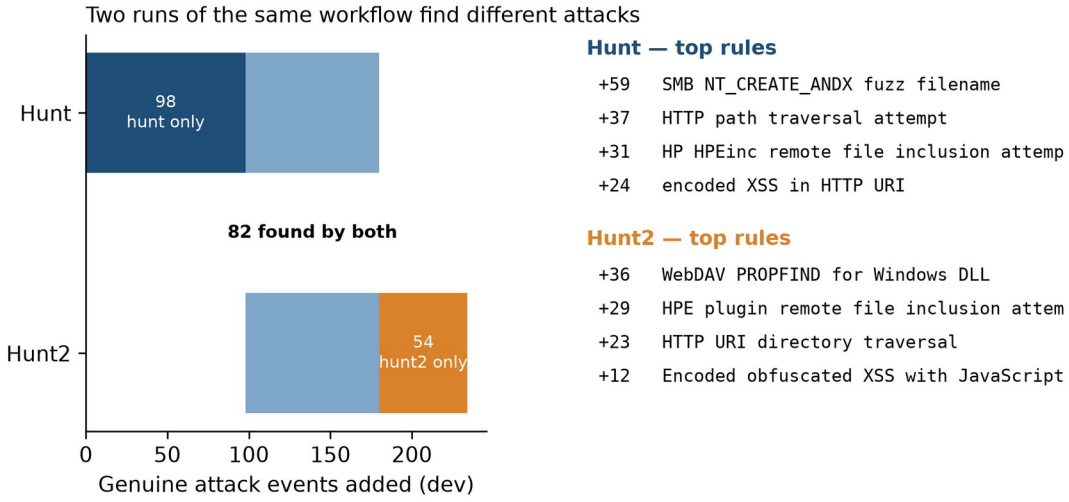


Fig. 5-11 : Genuine development events found by the hunt and hunt2 conditions, and each condition's top rules

What a Signature IDS Could Have Detected

Every percentage so far is measured against all labelled TCP/UDP attack events. Two narrower denominators test whether that is unfair to a signature IDS. Content-detectable events are those in which the attacker sent application data in a matching flow — more TCP bytes than headers alone account for, or any UDP datagram — so a content rule could in principle see them; events without data, such as SYN probes, can only be caught by rate or topology logic. (The TCP PSH flag could not be used: the traffic generator often sends data without it.) Ever-detected events are those caught by at least one configuration run in this project: the baseline, the all-rules-enabled control or a genuine AI rule. It is a practical, and circular, ceiling.

Measure	Development (6 captures)	Hold-out (1, 3, 6)
TCP/UDP attack events	14,935	4,634
Content-detectable	11,783 (78.9%)	3,495 (75.4%)
Ever-detected	4,999 (33.5%)	1,469 (31.7%)
Baseline v2: all / content / ever	24.9% / 23.7% / 74.4%	29.5% / 28.5% / 93.1%
Plain genuine gain, pts: all / content / ever	+0.08 / +0.08 / +0.24	+0.11 / +0.14 / +0.34

Research genuine gain, pts: all / content / ever	+0.33 / +0.42 / +1.00	+0.28 / +0.37 / +0.88
Hunt genuine gain, pts: all / content / ever	+1.21 / +1.49 / +3.60	+1.53 / +1.92 / +4.83
Hunt2 genuine gain, pts: all / content / ever	+0.91 / +1.15 / +2.72	+1.06 / +1.34 / +3.34

Table 5-5: Detection against all, content-detectable and ever-detected attack events

Excluding events without data barely changes the picture: about three-quarters of attack events carried data, and the baseline still detects only 23.7% of those on the development captures. The gap is missing signatures, not traffic that no signature could see (Figure 5-12). Against the ever-detected ceiling the human baseline already reaches 74.4% (development) and 93.1% (hold-out), and the hunt rules add +3.60 and +4.83 points of it — a more substantial share of what the project could catch than of what the dataset contains.

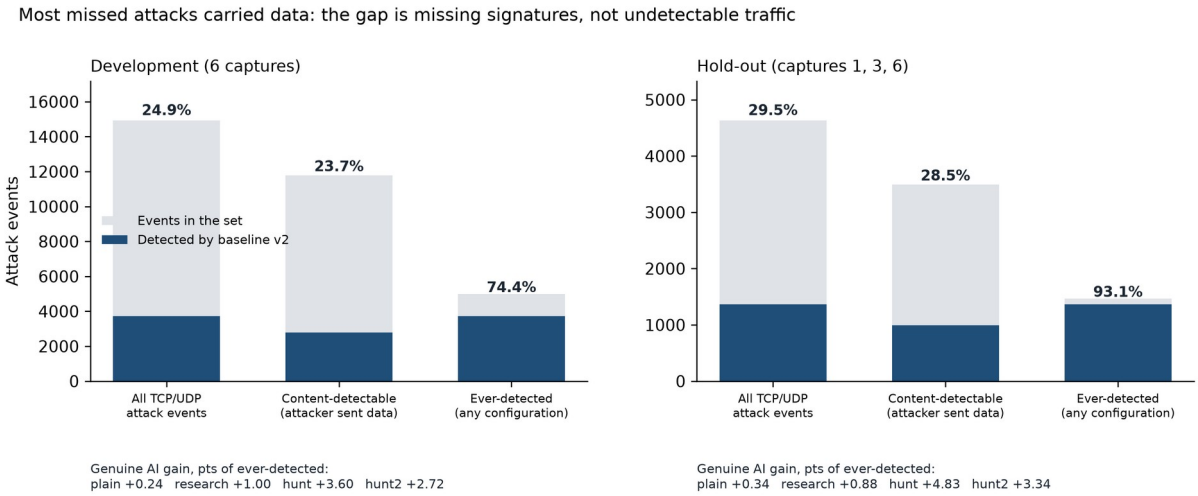


Fig. 5-12 : Baseline v2 detection against all, content-detectable and ever-detected attack events

Why the Agent Did Not Write a Rule

Each hunt report ends with activity the model judged attack-like but left without a rule. Grouped by cause (Figure 5-13), these show where the remaining limits lie. A tooling gap — not knowing which keywords this Suricata version supports — led to guessed keywords and raw-byte fall-backs; the keyword tool closed it. A rule-language limit: high-entropy DNS names suggesting tunnelling or generated domains need a statistic over the name, which a plain signature cannot compute; Suricata's lua keyword could, but the agent cannot write Lua scripts. Our own design choices: randomised POP3, IMAP and SMTP sessions can only be separated from legitimate users by rate or source, but the procedure forbids pivoting on addresses and the gate cannot test those protocols, so the model, reasonably, did not guess. DCE/RPC bind fuzzing with random interface versions would need a raw byte test on the bind header, which the model avoided rather than write a broad port rule. Finally, some omissions were

correct judgement: SNMP community probing was already covered by deployed rules, and lab-specific or over-narrow candidates were dropped.



Fig. 5-13 : Attack-like activity the hunt reports left without a rule, grouped by cause

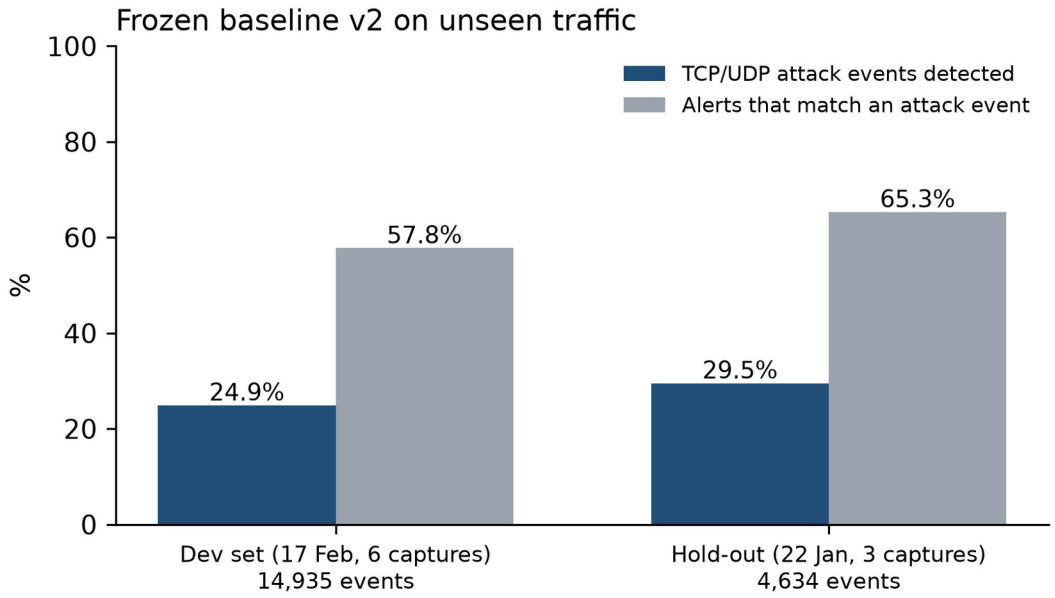


Fig. 5-14 : Frozen baseline v2 on the development captures and on the hold-out captures

Context Experiment (Experiment 1)

The research question asks what context the model needs. Experiment 1 compares the six full-context UNSW-NB15 runs, in which the model could query every event type, with runs on the same captures in which it could see alert events only (Section III). Three captures are repeated in the alerts-only condition (1, 3 and 10), and both conditions are scored with the same ground-truth method.

Measure (captures 3, 1, 10)	Full context	Alerts only
Categories identified per run (main)	3, 6, 5	7, 5, 5
Categories identified, union of runs (of 9)	6	7
TCP/UDP GT events covered by findings	1,117 of 6,753 (16.5%)	1,892 of 6,753 (28.0%)
Outcome statements: total (evidence-backed)	53 (52)	15 (9)
Rules written / fired on replay	8 / 8	10 / 9
Rule alerts matching GT events	37 of 123 (30%)	462 of 652 (71%)
Rule alerts on non-attacker hosts	0	0
TCP/UDP detection: stock + CIC → with condition's rules	8.8% → 8.9%	8.8% → 12.2%
Turns	44, 75, 62	57, 48, 60
Duration	10, 12, 9 min	10, 7, 8 min
Input tokens (three runs)	12.9 million	9.0 million

Table 5-6: Experiment 1 — full sensor context versus alerts only (DeepSeek V4 Pro, UNSW-NB15)

The result runs against the natural expectation that more context produces better triage. Without context the model was not worse at finding attacks: across the three captures it identified seven categories against six, and its findings covered 28.0% of TCP/UDP ground-truth events against 16.5%. Its rules added more detection (8.8% to 12.2%, against 8.8% to 8.9%), largely because two of them target portmapper sweeps, while the full-context runs on these captures happened to write narrower rules for individual compromises. Neither condition's rules fired on a benign host.

Context changed the depth of the triage rather than its breadth. With every event type available, the model established what each attack achieved — 59 password guesses answered '230 Login successful', 2.25 MB returned by a Heartbleed-vulnerable server, the byte counts of a reverse shell — and made 53 such outcome statements, all but one backed by evidence. Restricted to alerts, it produced broader, signature-organised reports with 15 outcome statements, six of which state that success cannot be confirmed from alert data alone. For an analyst the second question — did the attack succeed — decides the response, so context is what makes the triage actionable rather than what makes it complete (Figure 5-15).

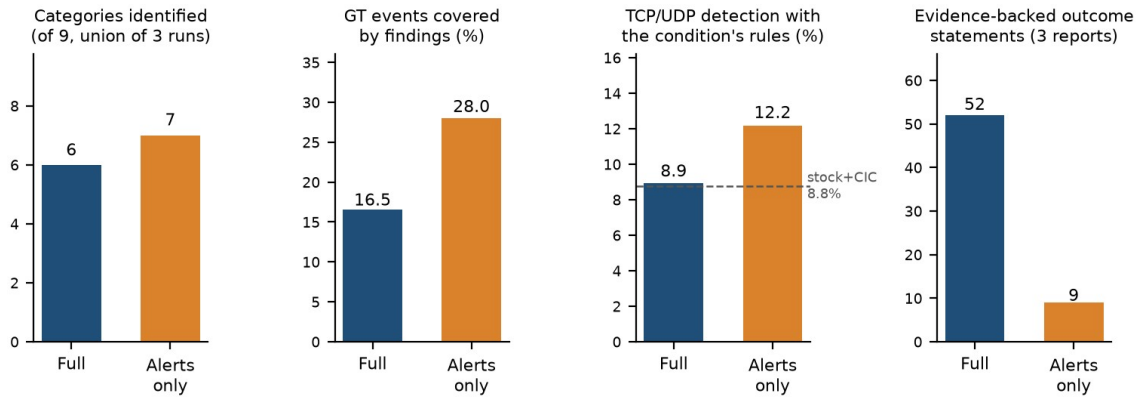


Fig. 5-15 : Experiment 1 — breadth of detection and depth of outcome evidence with full sensor context and with alerts only

Three caveats limit the result. Each condition was run once per capture, so run-to-run variation, which Section V shows to be substantial, is not separated from the effect of context. 'Alerts only' is not context-free: Suricata alert records carry their own per-flow byte and packet counters, which the alerts-only runs used. One alerts-only rule (SID 7700004) re-authored the CIC Heartbleed rule, as the full-context unsw-5 run had, so the ten rules are nine new detections; its alerts duplicate the baseline and do not contribute to the gain. And the findings measure rewards broad reports, which favours the enumerative style the alerts-only condition produced. The outcome-statement count is an automatic pattern match over the final reports and is indicative rather than exact. The designed context bundle (source history, asset context, rule history) was not built and remains future work.

Threats to Validity

Label leakage. The transcript audit found that the Thursday and Friday CIC-IDS2017 runs listed the ground-truth directory, whose file names name the attacks (for example '…Afternoon-DDos' and '…PortScan'); their attempts to read the files returned only the first rows, all benign. One Tuesday DeepSeek V3 run read the Tuesday ground truth from the project's notes (triage-2026-09-04-03-07-23.txt). The two earlier V3 runs did not read the notes but did not complete (turn limit), and Claude's run was not logged, so no completed Tuesday triage can be shown to be independent of the labels. The Tuesday rules are unaffected — ground-truth replay scored them (SID 9000004: 574 of 574 alerts on SSH-Patator flows; SID 9500001: 185 of 185 on FTP-Patator) — but the Tuesday triage finding itself must be treated as label-informed, although the interim report presented it as independent. Access is now restricted (Section III), and no UNSW-NB15 run attempted it.

Label quality and matching. CIC-IDS2017 labels contain documented errors (Engelen, Rimmer & Joosen 2021; Lanvin et al. 2023). About 300 alerts from the Wednesday slow-HTTP rules, on long, slow connections, and 334 from the FTP rule SID 7600004 could not be joined to any labelled flow. The matching rules are heuristics; their tolerances and exclusions are stated above so that the results can be reproduced.

Incomplete replay. The CIC-IDS2017 capture files were removed from the working environment to free storage for the UNSW-NB15 evaluation. The Tuesday and Thursday captures were re-obtained and replayed; the Wednesday and Friday rules are scored from the validation logs recorded during triage, and six rules whose files were never loaded could not be scored.

Sample size and provenance. Most conditions were run once, so run-to-run variation — visible in the split verdicts — is not quantified. The ARES session transcript was not exported, and there is no human-analyst baseline performing the same task.

Replay variance and attribution. Suricata replays of the same capture are not identical, so any comparison across replays mixes rule effects with timing effects. Round results are therefore attributed within a single replay; the earlier replay-versus-stored figures are superseded.

Rule classification. The topology, behavioural, signature and dataset-indicator classes were assigned from each rule's text, not from its alerts; a different reader might classify a borderline rule differently. The class of every rule is published with its reason (docs/results/round_rule_classes.json).

Hunt condition design. The hunt condition differs from the triage conditions in two ways at once — its procedure and the absence of the attacker range from its prompt — so the effect of each cannot be separated. Its known-good test relies on the testbed description and cannot test protocols that only external hosts use.

Reasoning carried across turns. Until the provider fix described in Section IV, the DeepSeek adapter discarded the model's reasoning whenever it answered in thinking mode, so the reasoning was neither logged nor returned on the next turn. Every earlier DeepSeek run — CIC-IDS2017, the first UNSW-NB15 runs, Experiment 1, the plain and research rounds and hunt capture 3 — ran with this defect; most of those runs also wrote visible text, which was carried, so they were affected less, but by how much cannot be measured after the fact. The hunt condition's gain coincides with the fix: its capture-3 rules (before) added 6 events, its capture-1 and capture-10 rules (after) 175. The comparison between the

hunt and triage conditions is therefore confounded with the fix; re-running the plain and research conditions with reasoning carried would separate the two.

One run per capture. Each condition was run once per capture. The hunt and hunt2 conditions, which differ only by one tool, found largely different attacks (Figure 5-11), so run-to-run variation is of the same size as the differences between the hunt-style conditions; those differences should not be read as rankings.

Detectability proxy. Content-detectable is estimated from flow byte counts; very small payloads may be counted as header-only, and the ever-detected ceiling is defined by this project's own rule sets.

Summary

The analysis shows the AI triage engine generates rules that trigger significantly more on an attack set than standard community rules for the set of attacks tested in the evaluation data set. The Tuesday model comparison is compromised by label leakage, so model-agnosticism remains a property of the design rather than a demonstrated result. Raw data analysis uncovers coverage gaps that are not apparent from suppressed passes. The key engineering lesson is that it is important to design the tools that do the job as much as the capabilities of the model: Purpose-built tools for NDJSON aggregation turned a flop into a success. An example of the type of error that might occur, as described above, is V4 Pro removing effective rules based on an incorrect assumption about a state in an external system. Only DoS GoldenEye was never identified; Bot/ARES was found in a later run. Scored against ground truth, the AI rules raised CIC-IDS2017 category coverage from 8 to 12 of 14 and UNSW-NB15 TCP/UDP event detection from 9.2% to 15.5%, with 99.4% precision on the noisiest captures — but the same evaluation found invalid rules, a false-positive rule and misread traffic that the model had reported with confidence.

VI. Summary

This section brings together the findings of the work done so far.

The triage engine completed the analyst workflow on all four attack days. The model identified 13 of the 14 labelled CIC-IDS2017 attack categories (all but DoS GoldenEye) and 8 of the 9 UNSW-NB15 categories (all but Analysis), on a dataset it had never been tuned for.

The models wrote 32 rule definitions on CIC-IDS2017 and 18 on UNSW-NB15. Scored against ground truth, the AI rules raised CIC-IDS2017

category coverage from 8 to 12 of 14 categories and UNSW-NB15 TCP/UDP event detection from 9.2% to 15.5%, never alerting on a benign UNSW host; on the noisiest CIC captures 99.4% of their label-matched alerts fell on attack flows, against 6.1% for the stock rules. The rule audit also found three rules that do not load, one that can never fire, one false-positive rule, and two effective rules deleted by the model. A further 53 rules were written in the Experiment 3 rounds on the strengthened baseline.

The entire process of detection is rule-based and is open for inspection. Safety asymmetry that makes it impossible to suppress autonomously is in place; the model proposes suppressions, but cannot do it without human consent, and hasn't been formally tested by deliberate adversarial probing. In one UNSW-NB15 run the model proposed suppressing rules that were detecting a real exploit; because the proposal required approval, the sensor was not blinded.

Several findings that have relevance beyond the scope of the project. More than the selection of the model, tool design is important: A model that did not work at all but was made a success by the investment in a purpose-built aggregation tool holds the promise that investment in more capable models can be replaced by investment in better tools. Also, raw data analysis reveals coverage gaps that cannot be detected by suppressed passes and motivates the idea of a two-pass method for production deployments. Third, rules written on one UNSW-NB15 capture fired on others; the apparent transfer of CIC-authored rules to UNSW-NB15 rests on two topology-style rules and is withdrawn (Section V). Against the strengthened baseline, rules keyed to a single exploit instance did not transfer, while technique-level hunt rules did. Suppose models hallucinate and make confident but false statements about external system states. For example, when effective rules are deleted based on a hallucinated premise, safeguards are important in any autonomous rule-management workflow. Finally, an agent's self-validation is only as good as its environment: where the configuration did not load the model's rule files, its validation runs reported success while testing nothing.

Strong baselines change the question. Tuning the human-written rules — re-enabling disabled families by service — raised UNSW-NB15 detection from 9.2% to 25.7%, more than any AI condition added. Against that baseline the triage-driven conditions' genuine contribution was a fraction of a point, and most of their apparent gain came from topology rules that only work because of how the dataset was built. Measuring only what the AI rules alone detected, and classifying every rule, was necessary to see this.

Replicating practice helps — once the agent keeps its own reasoning. Encoding the detection-engineering lifecycle and an engine-enforced test gate eliminated topology rules. With the model's reasoning carried across turns, the same procedure produced technique-level rules that added +1.21 points on the development captures and +1.53 on the hold-out, with no alert on benign traffic; rules keyed to single exploit instances did not transfer.

Knowledge of the rule language is not the bottleneck. A keyword reference removed every rule-loading failure but did not raise detection; what a run detects depends on which attack families it chooses to hunt, which varies between runs. Most missed attacks carried data, so the remaining gap is signatures nobody has written, and the limits the agent reported are mostly in the rule language and in our own safety choices.

Answer to the research question. The evidence supports a split answer. For triage, yes: with sensor-available context a large language model carries out the alert-handling workflow of a rule-based IDS end to end, identifying 13 of 14 CIC-IDS2017 and 8 of 9 UNSW-NB15 attack categories and establishing whether attacks succeeded. Experiment 1 shows what the context buys: alert events alone were enough to identify attacks as broadly, but flow and application-layer context is what allowed the model to establish outcomes — the question that decides an analyst's response. For extending detection, the answer is partial: the model's rules clearly extend stock ET Open, but against a human baseline strengthened the way a detection engineer would strengthen it, triage-driven rule writing added only a fraction of a percentage point of genuine detection — counted from events no other rule caught, with topology rules removed. Replicating detection-engineering practice, with the model's reasoning carried across turns, added 1.2 to 1.5 points that held on untouched traffic with no false alarms on benign captures: real, but modest against the roughly 70% of attack events every ruleset still misses, and confounded with a mid-round fix to the model adapter; a second run of the same workflow with a keyword reference added a similar, slightly smaller gain from largely different attacks. The model cannot replace the analyst unsupervised: its errors are confident and consequential, so rule changes that reduce detection must stay under human approval and every authored rule needs independent replay.

VII. Future Work

This section describes the work that remains to be done to finalize the project. The remaining work is divided into three categories: formal experimentation, system evaluation, and production architecture.

Experiment 2 has since been completed and Experiment 1 carried out in reduced form (Section V); the text below records their status and the remaining work.

Experiment 1: Does the context make a difference? Status: a reduced form (alerts only versus full sensor context) is reported in Section V; the full context bundle below remains future work. The current system supplies the model with all of the event types that are present in eve.json (alert, DNS, TLS, HTTP, flow records), but the full context bundle of the design (Section III) (30-day source history, destination asset context, activity in a + or – 5-minute window, and the history of firing rules) is not yet implemented. This bundle will allow for the creation of a controlled experiment, comparing the accuracy of triage based on just alert data to the accuracy of triage based on the full context bundle, to see if there is a measurable difference in the detected result.

Experiment 2: Quantitative evaluation of rules. Completed; see Section V, Ground-Truth Evaluation. Remaining: re-running the Wednesday and Friday rules against their captures so that every rule is scored from a single consistent replay.

Experiment 3: Convergence testing. If the volume of alerts decreases across N triage cycles while the suppression and detection loop is active, it indicates that the system improves at detecting attacks over time without machine learning. Carried out in revised form as repeated AI rule-writing rounds on the strengthened UNSW-NB15 baseline (Sections IV and V): instead of alert volume on benign traffic, each round measures the detection the AI rules add. Remaining: the hunt condition's captures 1 and 10, and a technique-level variant of the hunt procedure.

Human analyst baseline. A team member triaging one UNSW-NB15 capture by hand, timed and with the same tools, and scored with the same scripts, would allow a direct comparison of human and model performance.

Controlled model comparison. Two providers triaging the same capture in the locked-down environment (Section III), to test the model-agnosticism claim properly.

Environment-aware validation. The rule-validation tool should load any rule file the model has just written, so that self-validation always tests the new rules (see Rule Audit).

Technique-level hunting. A variant of the hunt profile that requires rules on the technique — over-long parameters instead of exploit filler, failure rates for scans and brute force — and that tests each rule on a second capture before it is saved.

Separate the workflow from the fix. Re-run the plain and research conditions with the model's reasoning carried across turns, so that the hunt workflow's own contribution can be measured.

Several runs per capture. Because runs of the same procedure find different attacks, repeat each condition and keep the union of genuine rules, measuring both the spread and the combined gain.

Statistical and rate rules. Allow Suricata Lua rules for properties such as DNS label entropy, and extend the test gate to rate rules — for example, a threshold that must never trip for any internal client.

Production deployment architecture: The current system uses an offline PCAP-replay mode. If this were extended to live deployment, it would need a few additions to the architecture:

- **Time-windowed alert batching:** Instead of processing the entire pcap file, the system would collect alerts over a period of time (5-15 minutes) and initiate a triage process for each group of alerts. This enables the current batch system to be transformed into a near-real-time continuous operation.
- **Automated triage scheduling**: Triage runs would now be automatically triggered on the webapp based on a threshold (either number of alerts or time) rather than manually.
- **Live rule reloading**: The Unix control socket on Suricata allows for reloading rules without engine restart (suricatasc -c reload-rules). The AI-generated rules would be deployed instantaneously to the running sensor, eliminating the detection window for minutes, not hours or days, since these rules are scheduled to be updated.
- **Multi-model, multi-sensor routing:** Usually, several sensors are deployed at the boundaries of the enterprise. The triage engine would forward alerts from each sensor to the corresponding model, depending on traffic volume, criticality level, cost, etc., for instance, using a lower-cost model to process routine alerts and a more powerful model for deep investigation of high-severity alerts.
- **SOC analyst dashboard:** Web-based dashboard display of suppress proposals to the human system for approval, including a full reasoning chain for each proposal. This makes the safety asymmetry (Section III) happen in a production chain.

Optimization for a large number of performances: The primary reason for such high per-query latency of the current system is that it scans the entire eve.json file for every tool call. At enterprise scale, the

following three optimisations would significantly reduce the time required to do the triage:

- pre-splitting eve.json by type of event (alerts, HTTP, flow, TLS, DNS), so that each query is only required to scan the subset of the JSON file that fits its type.
- pre-computing all big-picture statistics (alerts per signature, per source IP, per destination) and providing them as the first triage turn, which saves 3–5 model turns per run on average, since many alerts can be discarded after the first turn (e.g., checksum validation alerts that make up 98.8% of the raw alerts received on Wednesday).
- implementing a "raw minus capture artefacts" tier that removes decoder noise (e.g., checksum validation alerts) from the raw data, and preserves everything else relevant to threats. The changes are not to the triage engine or the model, but simply a preprocessing optimization that decreases the amount of data fed into the same agentic loop.

The same agentic loop, tools, and prompts can be used on live-captured alerts as part of the triage engine deployment transition. This is by design, since the model uses tool calls to communicate with the data, and switching from a file-backed tool to a database-backed one is invisible to the model.

References

Anderson, JP 1980, *Computer security threat monitoring and surveillance*, Technical report, James P. Anderson Company, Fort Washington, Pennsylvania. http://csrc.nist.gov/publications/history/ande80.pdf

Aljanabi, M, Ismail, MA & Ali, AH 2021, 'Intrusion detection systems, issues, challenges, and needs', *International Journal of Computational Intelligence Systems*, vol. 14, no. 1, pp. 560–571. **https://doi.org/10.2991/ijcis.d.210105.001**

Anthropic 2025, 'Tool use and agentic workflows', *Claude Documentation*, viewed 15 September 2026, https://docs.anthropic.com/en/docs/build-with-claude/tool-use

Atkinson, J 2020, 'Capability abstraction', SpecterOps blog, 6 February, viewed 4 October 2026, https://specterops.io/blog/2020/02/06/capability-abstraction/

Axelsson, S 2000, 'The base-rate fallacy and the difficulty of intrusion detection', *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205. https://doi.org/10.1145/357830.357849

Buczak, AL & Guven, E 2016, 'A survey of data mining and machine learning methods for cyber security intrusion detection', *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

Denning, DE 1987, 'An intrusion-detection model', *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232. https://doi.org/10.1109/TSE.1987.232894

di Filippo, L, Bardhi, E, Agiollo, A, Palma, A, Bonomi, S & Kuipers, F 2026, 'Beyond the syntax: do security experts trust LLMs for NIDS rule engineering?', arXiv preprint arXiv:2607.05916.

Dragos 2023, 'Improving long tail analysis using Neighborhood Keeper', Dragos blog, 27 April, viewed 4 October 2026, https://www.dragos.com/blog/using-neighborhood-keeper-for-long-tail-analysis/

Engelen, G, Rimmer, V & Joosen, W 2021, 'Troubleshooting an intrusion detection dataset: the CICIDS2017 case study', in 2021 IEEE Security and Privacy Workshops (SPW), pp. 7–12. https://doi.org/10.1109/SPW53761.2021.00009

Ferrag, MA, Maglaras, L, Moschoyiannis, S & Janicke, H 2020, 'Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study', *Journal of Information Security and Applications*, vol. 50, p. 102419. https://doi.org/10.1016/j.jisa.2019.102419

Fraunhofer FKIE 2026, 'UNSW-NB15', COMIDDS: Comprehensive Open Malware Intrusion Detection Dataset Store, viewed 4 October 2026, https://fkie-cad.github.io/COMIDDS/content/datasets/unsw_nb15/

Garcia-Teodoro, P, Diaz-Verdejo, J, Macia-Fernandez, G & Vazquez, E 2009, 'Anomaly-based network intrusion detection: Techniques, systems and challenges', *Computers & Security*, vol. 28, no. 1, pp. 18–28. https://doi.org/10.1016/j.cose.2008.08.003

Hadi, HJ, Yasmin, R & Shoker, A 2026, 'GenTI: Benchmarking LLMs for Autonomous IDPS Rule Generation for Unseen Attacks', arXiv preprint, arXiv:2606.05844.**https://doi.org/10.48550/arXiv.2606.05844**

Ji, Z, Lee, N, Frieske, R, Yu, T, Su, D, Xu, Y, Ishii, E, Bang, YJ, Madotto, A & Fung, P 2023, 'Survey of hallucination in natural language generation',

ACM Computing Surveys, vol. 55, no. 12, art. 248. https://doi.org/10.1145/3571730

Khraisat, A, Gondal, I, Vamplew, P & Kamruzzaman, J 2019, 'Survey of intrusion detection systems: techniques, datasets and challenges', *Cybersecurity*, vol. 2, no. 1, p. 20.https://doi.org/10.1186/s42400-019-0038-7

Ko, C, Ruschitzka, M & Levitt, K 1997, 'Execution monitoring of security-critical programs in distributed systems: A specification-based approach', in *Proceedings 1997 IEEE Symposium on Security and Privacy*, pp. 175–187, IEEE. https://doi.org/10.1109/SECPRI.1997.601332

Lanvin, M, Gimenez, PF, Han, Y, Majorczyk, F, Mé, L & Totel, É 2023, 'Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes', in Risks and Security of Internet and Systems (CRiSIS 2022), Lecture Notes in Computer Science, vol. 13857, Springer, pp. 18–33. https://doi.org/10.1007/978-3-031-31108-6_2

Li, J, Chai, Y, Du, L, Duan, C, Yan, H & Gu, Z 2025, 'GRIDAI: generating and repairing intrusion detection rules via collaboration among multiple LLM-based agents', arXiv preprint arXiv:2510.13257.

Liao, HJ, Lin, CHR, Lin, YC & Tung, KY 2013, 'Intrusion detection system: A comprehensive review', *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24. https://doi.org/10.1016/j.jnca.2012.09.004

Mitra, S, Neupane, S, Duclos, M, Mittal, S, Piplai, A, Rahman, MR, Zieglar, E & Rahimi, S 2025, 'FALCON: autonomous cyber threat intelligence mining with LLMs for IDS rule generation', arXiv preprint arXiv:2508.18684.

Moustafa, N & Slay, J 2015, 'UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)', in 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, pp. 1–6. https://doi.org/10.1109/MilCIS.2015.7348942

Moustafa, N & Slay, J 2016, 'The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set', Information Security Journal: A Global Perspective, vol. 25, no. 1–3, pp. 18–31.

OISF 2026, *Suricata 8.0.6 documentation*, Open Information Security Foundation, viewed 15 September 2026, https://docs.suricata.io/en/suricata-8.0.6/.

Palantir 2017, Alerting and detection strategy framework, GitHub repository, Palantir Technologies, viewed 4 October 2026, https://github.com/palantir/alerting-detection-strategy-framework

Paxson, V 1999, 'Bro: A system for detecting network intruders in real-time', *Computer Networks*, vol. 31, no. 23, pp. 2435–2463. https://doi.org/10.1016/S1389-1286(99)00112-7

Proofpoint 2026, *Emerging Threats Open ruleset*, Proofpoint Inc., viewed 15 September 2026, https://rules.emergingthreats.net/open/ .

Rehman, MU, Piplai, A & Kantarcioglu, M 2026, 'RuleAutoPilot: synthesizing deployable Suricata rules from network traffic', arXiv preprint arXiv:2609.16231.

Ring, M, Wunderlich, S, Scheuring, D, Landes, D & Hotho, A 2019, 'A survey of network-based intrusion detection data sets', Computers & Security, vol. 86, pp. 147–167. https://doi.org/10.1016/j.cose.2019.06.005

Roesch, M 1999, 'Snort: Lightweight intrusion detection for networks', in *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pp. 229–238. https://www.usenix.org/legacy/event/lisa99/full_papers/roesch/roesch.pdf

Scarfone, K & Mell, P 2007, *Guide to intrusion detection and prevention systems (IDPS)*, NIST Special Publication 800-94, National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-94

Schick, T, Dwivedi-Yu, J, Dessì, R, Raileanu, R, Lomeli, M, Hambro, E, Zettlemoyer, L, Cancedda, N & Scialom, T 2023, 'Toolformer: language models can teach themselves to use tools', in Advances in Neural Information Processing Systems, vol. 36.

Sekar, R, Gupta, A, Frullo, J, Shanbhag, T, Tiwari, A, Yang, H & Zhou, S 2002, 'Specification-based anomaly detection: a new approach for detecting network intrusions', in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pp. 265–274. https://doi.org/10.1145/586110.586146

Sharafaldin, I, Habibi Lashkari, A & Ghorbani, AA 2018, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', in *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pp. 108–116. https://doi.org/10.5220/0006639801080116

Sommer, R & Paxson, V 2010, 'Outside the closed world: On using machine learning for network intrusion detection', in *2010 IEEE Symposium on Security and Privacy*, pp. 305–316, IEEE. https://doi.org/10.1109/SP.2010.25

Vaswani, A, Shazeer, N, Parmar, N, Uszkoreit, J, Jones, L, Gomez, AN, Kaiser, Ł & Polosukhin, I 2017, 'Attention is all you need', in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008. https://doi.org/10.48550/arXiv.1706.03762

Yao, S, Zhao, J, Yu, D, Du, N, Shafran, I, Narasimhan, K & Cao, Y 2023, 'ReAct: synergizing reasoning and acting in language models', in Proceedings of the 11th International Conference on Learning Representations (ICLR 2023).

Zhang, X, Du, X, Chen, H, He, Y, Niu, W & Li, Q 2025, 'Automatically generating rules of malicious software packages via large language model', in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 734–747, IEEE. https://doi.org/10.1109/DSN64029.2025.00072

Appendix A: Turnitin Similarity Report

Appendix B

<table>
<tr><td colspan="2">Reference to one or more AI tools. :
AI Tool: claude</td></tr>
<tr><td>List all inputs(prompts) you have used.
This is the prompt we give the AI at the start of every triage run. It's in src/prompts.py:

"You are an IDS triage analyst. You have access to Suricata's eve.json alert log. Your job is to investigate alerts, determine what is happening, and take action. Follow this procedure: (1) Big picture - count all alerts to understand the landscape, (2) Categorise - group alerts by type and severity, (3) Investigate - drill into each category, (4) Deep-dive - examine specific flows and packets, (5) Act - write detection rules for confirmed threats or propose suppressions for false positives, (6) Report - summarise findings with</td><td>Outputs

AI Outputs

Tuesday Triage Output (DeepSeek V3)

- Findings: FTP brute force (172.16.0.1 → 192.168.10.50, 11,891 failures, 1 success with password "1234"), SSH brute force (~2,980 paramiko flows)
- Rules written: 5 detection rules (SID 9000001-9000005) in rules/generated/tuesday_bruteforce.rules
- Full triage log: triage-2026-09-04-03-07-23.txt (44 turns of investigation) [Note: this run read the Tuesday ground truth from the project notes; see Section V, Threats to Validity.]

Tuesday Triage Output (Claude Opus 5)

- Findings: Same attacks identified independently
- Rules written: 2 campaign threshold rules (SID 9500001-9500002) in rules/generated/bruteforce_campaigns.rules

Wednesday Triage Output (two passes: DeepSeek V3 tuned, DeepSeek V4 Pro raw)

- Pass 1 findings: Slowloris/Slowhttptest DoS (167K+ requests),</td></tr>
</table>

evidence." 2. Tool Descriptions Each of the 8 tools has a description the AI reads to know what it can do. Examples: - query_eve_json: "Run a jq query against an eve.json file. Use this to investigate specific IPs, signatures, or traffic patterns." - count_by: "Aggregate NDJSON data by any field. Never use group_by, unique, min_by, max_by, use this tool instead." - write_file: "Write content to a file. Use this to create Suricata detection rules in rules/generated/." 3. Per-Run Input Each triage run, the AI receives: "Triage the alerts in logs/<run-name>/eve.json" - that's the only user message. Everything else the AI does on its own using tools.	Heartbleed probing (CVE-2014-0160) - Pass 2 findings: Validated AI rules (SID 3000001 fired 187 times), found bug in SID 3000003 - Pass 3 findings: Hulk GET flood gap discovered, new rule written - Rules written: 4 detection rules (SID 3000001-3000004) in rules/generated/wednesday_slowloris.rules - Suppress proposals: 4 signatures recommended for suppression (required human approval) - Full triage log: triage-2026-09-04-04-10-26.txt [Correction: Wednesday had two passes, not three — DeepSeek V3 tuned (SID 3000001–3000003) and DeepSeek V4 Pro raw (SID 3000004 and the Heartbleed rules). See Section IV.] [Addition: later outputs include the Thursday and Friday triage (DeepSeek V4 Pro), the ARES botnet rules (21 September) and six UNSW-NB15 triage runs; all transcripts are in the repository.]
Reflect on how you used these AI tools, including how they supported your assignment, the limitations you encountered, and how you modified the AI-generated output. Please provide as much detail as possible, and you may include screenshots or highlight the parts you revised.	**Response**: This project has two different applications of AI: **Use 1: AI as the subject of research (triage engine)** Our project focuses on creating an AI triage engine that can streamline the tasks of a human IDS analyst. Constructed a pipeline using Python that takes Suricata alerts and feeds them into language model APIs (Claude Opus 5 and DeepSeek V3). The AI gets one system prompt to describe a 6-step triage process and 8 tools, which it can use to query alert data with jq, count and aggregate alerts, read and write Suricata configuration files, and run Suricata to check Suricata detection rules. The AI is not provided with answers or the information that it should seek. It gets only “Triage the alerts in this file” and on its

	own determines which alerts to examine, which questions to ask the data, and which rules to create. For instance, during brute force attacks on Tuesday, DeepSeek V3 found that only one out of 11,891 FTP logins was successful, found the password to be 1234, checked for post login exfiltration and generated a stateful 2 stage attack detection rule that would flag repeated failed login attempts and then alert on success by the same host. U**se 2: Claude Code (Development tool)** To help write the Python engine code, to help in debugging the Suricata rule syntax, and to help draft documentation, Claude Code (Anthropic's CLI tool) was used. The team reviewed, tested and modified all code before use. **How AI supported the assignment** The triage engine was able to detect all confirmed attacks for Tuesday (FTP/SSH brute force) and Wednesday (Slowloris DoS, Hulk flood, Heartbleed) in the CIC-IDS2017 dataset. - AI-generated detection rules performed better than the stock community detection rules – SID 3000001 detected 187 Slowloris attacks vs 3 on the Emerging Threats rule (62× better). - AI found a detection hole (Hulk GET flood) not covered by the 52,647 rule in an ET Open ruleset and declared a new rule to catch 95 attacks The results of running two models on a single dataset were comparable: Both led to the same predictions, but in Claude's case, the model predicted more quickly and did not make any errors using tools, whereas in DeepSeek, the model was 10x more expensive and made similar errors using tools. [Qualification: the completed DeepSeek V3 run read the Tuesday ground truth from project notes; see Section V, Threats to Validity.] **Limitations encountered** 1. NDJSON aggregation failure — DeepSeek V3 failed on early runs because it attempted to work on jq operations (group_by, unique, min_by) that don't support streaming JSON (one object per line). It reached the 30 turn mark but never completed any triage. The answer is that we need to create our own count_by tool in Python. This was an eye opener that it is the tool design more than the choice of model — a cheaper one can be as good, or be as good. 2.. Cost considerations – Claude Opus 5 is roughly $3-5 per triage run and DeepSeek V3 is $0.30-0.50 per run. We didn't have the time to submit a student project for Claude, but were forced to use DeepSeek for iteration testing instead. 3. Safety risk — An AI that silently blinds the IDS. This was

	addressed by introducing a safety asymmetry: The AI can provide auto-deploy detection rules while suppressing proposals, preventing the sensors from seeing. The modifications were made to the AI-generated output. Any edits that were made to the AI-generated output. Detection rules were not randomly applied. All the AI-generated rules were syntax checked with Suricata -T ; three of them (SID 7600002, 7600005, 7600006) fail to load in Suricata 8.0.6 (Section V, Rule Audit). This was not an AI mistake; it's a Suricata limitation and therefore will not be corrected in the code base, but it will be documented in. This is due to the fact that the failure is a Suricata limitation, and not an AI error, so it will be left in the code base as is, but will be documented in THI Proposals that needed to be suppressed were not automatically suppressed, but were reviewed. The AI suggested that on the first Wednesday pass, 7 signatures should be suppressed and that some stream anomalies should be considered as symptoms of the DoS attack and not noise. We considered every proposal with regard to the attack ground truth before accepting. Our modification to combat AI failure was the count_by tool. If DeepSeek was not able to aggregate data, this does not mean that the model or the tool has been changed. This is an example of system modification instead of AI output modification. [Check: the sentence stating that DeepSeek was 10× more expensive contradicts the cost figures that follow (Claude is the more expensive model).] [Addition: Claude Code was also used to write the evaluation scripts (src/eval/), produce the figures, and draft sections of this final report; the team reviewed all output.]
Do you **keep a verifiable AI prompt history accessible** to your supervisor in weekly meetings?	Yes Yes. All AI prompts are version-controlled in the Git repository (src/prompts.py). Full triage session transcripts including every prompt, tool call, and AI response are saved as timestamped log files (triage-*.txt). AI-generated detection rules are preserved unedited in rules/generated/. The complete history is available for inspection at any weekly meeting.

	If No, provide explanations:

Appendix C

The complete source code, configuration files, and AI-generated detection rules are available at:

https://github.com/bigyan-siwakoti/Intrusion_Detection_System

Appendix D: AI-Authored Rule Audit

Every AI-authored rule definition, its authoring session, load status and replay evidence. CIC evidence: alerts, and label-matched alerts on attack / benign flows. UNSW evidence: alerts, ground-truth-matched alerts, and captures (of six) on which the rule fired. Generated from docs/results/rule_audit.csv.

SID	Data	Author / session	Status	Replay evidence	Notes
3000001	CIC	DeepSeek V3, Wednesday tuned (triage-2026-09-04-04-10-26.txt)	deleted	CIC: 336 alerts (55 attack / 1 benign)	deleted by V4 Pro on a false 'stock ET' claim; text preserved in transcript
3000002	CIC	DeepSeek V3, Wednesday tuned (triage-2026-09-04-04-10-26.txt)	deleted	CIC: 14 alerts (1 attack / 0 benign)	deleted by V4 Pro on a false 'stock ET' claim; text preserved in transcript
3000003	CIC	DeepSeek V3, Wednesday tuned (triage-2026-09-04-04-10-26.txt)	loads	CIC: 463 alerts (457 attack / 0 benign)	rewritten in place by V4 Pro (http.method -> raw TCP depth:8)
3000004	CIC	DeepSeek V4 Pro, Wednesday raw (triage-2026-09-15-11-26-28.txt)	loads	CIC: 188 alerts (188 attack / 0 benign)	—
7600001	CIC	DeepSeek V4 Pro, Thursday tuned (triage-2026-09-15-13-31-04.txt)	loads	CIC: 8 alerts (8 attack / 0 benign)	—
7600002	CIC	DeepSeek V4 Pro, Thursday tuned (triage-2026-09-15-13-31-04.txt)	fails to load	—	fails to parse: smb.command keyword unknown
7600003	CIC	DeepSeek V4 Pro, Thursday tuned (triage-2026-09-15-13-31-04.txt)	loads	CIC: 4 alerts (2 attack / 2 benign)	—
7600004	CIC	DeepSeek V4 Pro, Thursday tuned (triage-2026-09-15-13-31-04.txt)	loads	CIC: 768 alerts (0 attack / 434 benign)	false positive: fires on routine FTP uploads from every workstation, Tuesday and Thursday; all label-matched hits are BENIGN
7600005	CIC	DeepSeek V4 Pro, Thursday tuned (triage-2026-09-15-13-31-04.txt)	fails to load	—	fails to parse: smb.command keyword unknown
7600006	CIC	DeepSeek V4 Pro, Thursday tuned (triage-2026-09-15-13-31-04.txt)	fails to load	—	fails to parse: malformed dcerpc.iface
7600007	CIC	DeepSeek V4 Pro, Thursday raw (triage-2026-09-15-13-30-50.txt)	loads	CIC: 244 alerts (0 attack / 244 benign)	fires only on compromised host .8; labels mark these flows BENIGN (CIC labels only 36 Infiltration flows) - disputed
7600101	CIC	DeepSeek V4 Pro, Wednesday raw (triage-2026-09-15-11-26-28.txt)	loads, cannot fire	—	loads but can never fire: tls.invalid_encrypted_heartbeat not registered in Suricata 8.0.6
7600102	CIC	DeepSeek V4 Pro, Wednesday raw (triage-2026-09-15-11-26-28.txt)	loads	UNSW: 339 alerts, 268 GT-matched, 6/6 captures	Heartbleed file never loaded in any Wednesday replay
7600103	CIC	DeepSeek V4 Pro, Wednesday raw (triage-2026-09-15-11-26-28.txt)	loads	UNSW: 287 alerts, 227 GT-matched, 6/6 captures	Heartbleed file never loaded in any Wednesday replay

7600201	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-3 triage	loads	UNSW: 53 alerts, 17 GT-matched, 4/6 captures	—
7600202	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-3 triage	loads	UNSW: 2 alerts, 2 GT-matched, 1/6 captures	—
7600203	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-3 triage	loads	UNSW: 3 alerts, 3 GT-matched, 3/6 captures	—
7600204	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-1 triage	loads	UNSW: 97 alerts, 33 GT-matched, 5/6 captures	—
7600205	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-1 triage	loads	UNSW: 10 alerts, 8 GT-matched, 4/6 captures	—
7600401	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-4 triage	loads	UNSW: 115 alerts, 101 GT-matched, 6/6 captures	—
7600402	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-4 triage	loads	UNSW: 98 alerts, 80 GT-matched, 6/6 captures	—
7600501	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-5 triage	loads	UNSW: 339 alerts, 268 GT-matched, 6/6 captures	re-authored copy of CIC rule Duplicate of 7600102.
7600502	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-5 triage	loads	UNSW: 287 alerts, 227 GT-matched, 6/6 captures	re-authored copy of CIC rule Duplicate of 7600103.
7600503	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-5 triage	loads	UNSW: 195 alerts, 170 GT-matched, 6/6 captures	—
7600504	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-5 triage	loads	UNSW: 511 alerts, 442 GT-matched, 6/6 captures	—
7600505	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-5 triage	loads	UNSW: 27 alerts, 26 GT-matched, 6/6 captures	—
7601001	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-10 triage	loads	UNSW: 4 alerts, 0 GT-matched, 2/6 captures	model skipped validation; replay-validated by the team
7601002	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-10 triage	loads	UNSW: 14 alerts, 10 GT-matched, 4/6 captures	model skipped validation; replay-validated by the team
7601003	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-10 triage	loads	UNSW: 4 alerts, 1 GT-matched, 4/6 captures	model skipped validation; replay-validated by the team
7601401	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-14 triage	loads	UNSW: 14 alerts, 10 GT-matched, 4/6 captures	duplicate logic Duplicate of 7601002.
7601402	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-14 triage	loads	UNSW: 27 alerts, 26 GT-matched, 6/6 captures	duplicate logic Duplicate of 7600505.
7601403	UNSW	DeepSeek V4 Pro, UNSW-NB15 unsw-14 triage	loads	UNSW: 45 alerts, 5 GT-matched, 4/6 captures	same purpose as 7601003 but broader (13 vs 1 hits on pcap 14)
9000001	CIC	DeepSeek V3, Tuesday triage (2 Sep; committed c14b002; no transcript kept)	loads	—	did not fire on Tuesday replay (stateful hostbits chain / thresholds never met)
9000002	CIC	DeepSeek V3, Tuesday triage (2 Sep; committed c14b002; no transcript kept)	loads	—	did not fire on Tuesday replay (stateful hostbits chain / thresholds never met)
9000003	CIC	DeepSeek V3, Tuesday triage (2 Sep; committed c14b002; no transcript kept)	loads	—	did not fire on Tuesday replay (stateful hostbits chain / thresholds never met)
9000004	CIC	DeepSeek V3, Tuesday triage (2 Sep; committed c14b002; no transcript kept)	loads	CIC: 575 alerts (574 attack / 0 benign)	—
9000005	CIC	DeepSeek V3, Tuesday triage (2 Sep; committed	loads	—	did not fire on Tuesday replay (stateful hostbits

		c14b002; no transcript kept)			chain / thresholds never met)
9500001	CIC	Claude Opus 5, Tuesday triage (CLI run, not logged; committed c14b002)	loads	CIC: 185 alerts (185 attack / 0 benign)	—
9500002	CIC	Claude Opus 5, Tuesday triage (CLI run, not logged; committed c14b002)	loads	CIC: 376 alerts (376 attack / 0 benign); UNSW: 4 alerts, 0 GT-matched, 2/6 captures	—
9500003	CIC	DeepSeek V4 Pro, Friday tuned (triage-2026-09-15-13-30-21.txt)	loads	—	—
9600001	CIC	DeepSeek V4 Pro, Friday raw (triage-2026-09-15-13-30-09.txt)	loads	CIC: 49 alerts (49 attack / 0 benign)	—
9600002	CIC	DeepSeek V4 Pro, Friday raw (triage-2026-09-15-13-30-09.txt)	loads	CIC: 17 alerts (15 attack / 2 benign)	—
9600003	CIC	DeepSeek V4 Pro, Friday raw (triage-2026-09-15-13-30-09.txt)	loads	CIC: 510 alerts (505 attack / 4 benign)	copy placed in Claude's bruteforce_campaigns.rules so it would load Duplicate of 9600001.
9600004	CIC	DeepSeek V4 Pro, Friday raw (triage-2026-09-15-13-30-09.txt)	loads	CIC: 17 alerts (16 attack / 1 benign)	copy placed in Claude's bruteforce_campaigns.rules so it would load Duplicate of 9600002.
9600005	CIC	DeepSeek V4 Pro, Friday validation run for Week 9 demo (21 Sep; no transcript kept)	loads	—	first ARES draft; file never loaded; superseded by 9700001-3
9600006	CIC	DeepSeek V4 Pro, Friday validation run for Week 9 demo (21 Sep; no transcript kept)	loads	—	first ARES draft; file never loaded; superseded by 9700001-3
9600007	CIC	DeepSeek V4 Pro, Friday validation run for Week 9 demo (21 Sep; no transcript kept)	loads	—	first ARES draft; file never loaded; superseded by 9700001-3
9700001	CIC	DeepSeek V4 Pro, Friday validation run for Week 9 demo (21 Sep; no transcript kept)	loads	CIC: 5 alerts (5 attack / 0 benign)	defined in two files (friday_botnet_c2.rules, bruteforce_campaigns.rules)
9700002	CIC	DeepSeek V4 Pro, Friday validation run for Week 9 demo (21 Sep; no transcript kept)	loads	CIC: 5 alerts (5 attack / 0 benign)	defined in two files (friday_botnet_c2.rules, bruteforce_campaigns.rules)
9700003	CIC	DeepSeek V4 Pro, Friday validation run for Week 9 demo (21 Sep; no transcript kept)	loads	CIC: 5 alerts (5 attack / 0 benign)	defined in two files (friday_botnet_c2.rules, bruteforce_campaigns.rules)

Table D-1: Inventory of AI-authored rule definitions